

Berliner Hochschule für Technik

Fachbereich VII: Elektrotechnik - Mechatronik - Optometrie

Bachelorarbeit

Vergleich verschiedener Dateisysteme für den SD-Karten-Einsatz in eingebetteten Systemen

Autor: Michael Kolorz

Matrikelnummer: 100359

Studiengang: Elektrotechnik (B.Eng.)

Schwerpunkt: Kommunikationstechnik

Erstprüfer: Prof. Dr. Peter Gober

Zweitprüfer: Prof. Dr. David Dietrich

Semester: Sommersemester 2026

Abgabedatum: 1. Oktober 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Projektübersicht	3
2	Hardware-Design und Schnittstellen	5
2.1	Einleitung	5
2.2	Idee und Schaltplan	5
2.2.1	Spannungsversorgung	6
2.2.2	USB-C	6
2.2.3	SPI-Interfaces	7
2.2.4	Programmier-Interfaces	7
2.2.5	VREF und ADCs/DACs	8
2.2.6	Weitere Peripherien	8
2.3	Routing und Design	8
2.3.1	Footprints	8
2.3.2	Design	9
2.3.3	Fertigung	9
2.3.4	Bestückung	10
2.3.5	Inbetriebnahme und Probleme	11
2.4	Fazit	12
2.5	Testhardware	13
3	SPI- und SD-Karten-Treiber	13
3.1	Embedded C++20	13
3.2	GPIO HAL	13
3.3	Debugging mit GDB	14
3.4	SPI	17
3.4.1	Anschlussbelegung	17
3.4.2	SPI-Peripherie des STM32G431	19
3.4.3	SPI Treiber	20
3.5	Initialisierung der SD-Karte in den SPI-Modus	21
3.6	Lese- und Schreiboperationen	23
3.6.1	Begriffsklärung Sektor, Block, Cluster, Page	23
3.6.2	SD-Karte als Blockspeicher	24
3.6.3	Lesefunktionen	24
3.6.4	Schreibfunktionen	25
3.6.5	Statusfunktionen	26
4	Dateisysteme	26
4.1	Einleitung	26
4.2	Layout eines Dateisystem	28

4.3	Design eines Dateisystems	28
4.3.1	Nicht-resiliente, blockbasierte Dateisysteme	28
4.3.2	Logging-Dateisysteme	29
4.3.3	Journaling-Dateisysteme	29
4.3.4	Copy-on-Write (COW) Dateisysteme	29
4.4	FAT-Dateisysteme	30
4.5	FatFs	32
4.5.1	Einleitung	32
4.5.2	Implementierung	32
4.6	LittleFS	35
4.6.1	Ziele von littlefs	35
4.6.2	Funktionsweise	35
4.6.3	Implementierung	38
4.6.4	littlefs FUSE wrapper	38
5	Vergleich littlefs, FatFs und direkter Blockadressierung	39
5.1	RAM- und Flashverbrauch	39
5.2	Versuch 1: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge	41
5.3	Versuch 2: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge und manipulierter Schreibfunktion	45
5.4	Versuch 3: Analyse von pseudo-randomisierter Zeichenfolge bei Anhang an Datei	48
5.5	Versuch 4: Verhalten der Dateisysteme bei Stromunterbrechungen	50
5.6	Versuch 5: Direkte Blockadressierung (NoFs)	54
6	Gesamtfazit	56
	Quellenverzeichnis	58

1 Einleitung

SD-Karten sind eine weit verbreitete Möglichkeit, um Daten auf eingebetteten Systemen zu speichern und zu lesen. Sie sind, verglichen mit Flash-Bausteinen, bei großer Speicherkapazität vergleichsweise günstig und benötigen auf einer Platine wenig Platz.

Im Rahmen dieser Bachelorarbeit wurde die Verwendung von SD-Karten an einem Mikrocontroller untersucht. Als Mikrocontroller kam dabei ein STM32G431 zum Einsatz, der über die SPI-Schnittstelle mit einer SD-Karte kommunizierte. Dafür wurde zunächst eine eigene Schaltung entwickelt und auf einer selbst entworfenen Platine umgesetzt. Neben der eigentlichen SD-Karten-Schnittstelle enthält die Platine weitere Schnittstellen und Peripherie, die für die Entwicklung und Untersuchung des Systems benötigt wurden.

Auf der Softwareseite wurden eigene SPI- und SD-Karten-Treiber in C++20 entwickelt. Diese bilden die Grundlage für die Kommunikation zwischen dem Mikrocontroller und der SD-Karte. Es wurden die für den Zugriff auf eine SD-Karte notwendigen Funktionen zum Lesen und Schreiben von Blöcken implementiert. Ein besonderer Fokus lag dabei auf der Verwendung der SD-Karte im SPI-Modus, da dieser eine vergleichsweise einfache Anbindung ermöglicht und SPI-Peripherien bei Mikrocontrollern sehr verbreitet sind.

Aufbauend auf der direkten Blockadressierung wurden anschließend verschiedene Dateisysteme betrachtet. Dazu wurden mit FatFs und littlefs zwei unterschiedliche Dateisysteme auf dem Mikrocontroller eingesetzt und an den zuvor entwickelten SD-Treiber angebunden. Dabei wurde insbesondere untersucht, welche Eigenschaften die beiden Dateisysteme im Vergleich zueinander besitzen. Neben dem benötigten Speicher auf dem Mikrocontroller wurden auch das Verhalten bei Schreiboperationen und die Zuverlässigkeit der Datenspeicherung betrachtet. Aufgrund von unerwarteten Ergebnissen bei ersten Tests wurden zusätzlich Zeitmessungen bei der Erstellung der Testdateien durchgeführt und die Ergebnisse visualisiert.

Ein Schwerpunkt der Arbeit lag auf der Untersuchung von Schreibfehlern und dem Verhalten des Systems unter verschiedenen Bedingungen. Hierzu wurden mehrere Versuche durchgeführt: Zunächst wurden Schreiboperationen mit bekannten Daten untersucht. Anschließend wurden die Schreibfunktionen des SD-Treibers gezielt verändert, um die Auswirkungen von Fehlern bei den unterschiedlichen Dateisystemen untersuchen zu können. Ein weiterer Versuch untersuchte das Verhalten der Dateisysteme bei randomisierten Daten und diente als Vorbereitung für den abschließenden Versuch, der das Verhalten der Dateisysteme bei unerwarteten Stromausfällen betrachtete.

1.1 Projektübersicht

Der zu dieser Bachelorarbeit gehörende Quellcode ist auf GitHub verfügbar unter <https://github.com/miko8278/stm32-SPI-SD-card> und wird dort in englischer Sprache geführt. Dadurch soll der Code einer breiteren Gruppe von Interessierten zugänglich sein.

Da im weiteren Verlauf dieser Arbeit mehrfach auf konkrete Dateien und Verzeichnisse des Softwareprojekts verwiesen wird, soll die folgende Übersicht als Navigationshilfe über die wichtigsten Dateien und die Struktur des Repositories auf GitHub dienen.

```

1  .
2  |-- code
3  |   |-- spi-sdcardwrite           # Hauptprojekt
4  |       |-- build                 # Build-Verzeichnis CMake
5  |       |-- build.sh              # Build-Skript
6  |       |-- CMakeLists.txt        # CMake-Projektkonfiguration
7  |       |-- FatFs                 # FatFs-Dateisystem extern
8  |       |   |-- diskio.c          # Mitgelieferte Beispiel diskio
9  |       |   |-- diskio.h
10 |       |   |-- ff.c               # Kernimplementierung von FatFs
11 |       |   |-- ffconf.h          # FatFs-Konfigurationsdatei
12 |       |   |-- ff.h              # Öffentliche FatFs-Schnittstelle
13 |       |   |-- ffsystem.c
14 |       |   |-- `-- ffunicode.c   # Codes für LFS, lange Dateinamen
15 |       |-- include                # Projektweite Header-Dateien STM
16 |       |-- linker
17 |       |   |-- STM32G431RBTX_FLASH.ld # Linker-Skript
18 |       |-- littlefs              # littlefs-Dateisystem extern
19 |       |   |-- lfs.c              # Kernimplementierung von littlefs
20 |       |   |-- lfs.h              # Öffentliche littlefs-Schnittstelle
21 |       |   |-- lfs_util.c
22 |       |   |-- lfs_util.h
23 |       |   |-- `-- LICENSE.md     # Lizenz von littlefs
24 |       |-- size
25 |       |   |-- size_fatfs.cpp     # Speicherverbrauch mit FatFs
26 |       |   |-- size_littlefs.cpp  # Speicherverbrauch mit littlefs
27 |       |   |-- size_nofs.cpp      # Speicherverbrauch ohne Dateisystem
28 |       |   |-- `-- sizes_allopt.txt # Ergebnisse der Größenmessungen
29 |       |-- src
30 |       |   |-- diskio.cpp         # Anbindung von FatFs an den SD-Treiber
31 |       |   |-- fatfs_time.cpp     # Zeitfunktion für FatFs
32 |       |   |-- GPIO_HAL.hpp       # Kleine GPIO-Hardwareabstraktion
33 |       |   |-- init_conf.hpp      # Initialisierungskonfiguration
34 |       |   |-- sdcarddriver.hpp    # SD-Karten-Treiber
35 |       |   |-- sdcardlittlefs.cpp # Anbindung von littlefs an den SD-Treiber
36 |       |   |-- sdcardlittlefs.hpp
37 |       |   |-- `-- spidriver.hpp  # SPI-Treiber
38 |       |-- startup_stm32g431xx.s  # Startup-Code des STM
39 |       |-- STM32G431xx.svd        # Beschreibung der MCU-Register für den Cortex-Debugger
40 |       |-- system_stm32g4xx.c     # Systeminitialisierungsdatei STM
41 |       |-- test
42 |       |   |-- fatfs_8.3_write_test.cpp # FatFs-Schreibtest mit 8.3-Dateinamen
43 |       |   |-- fatfs_write_test.cpp    # Schreibtest 1+2 für FatFs
44 |       |   |-- littlefs_write_test.cpp  # Schreibtest 1+2 für littlefs
45 |       |   |-- sd_test.cpp              # Haupttest des SD-Karten-Treibers
46 |       |   |-- `-- spi_test.gdb        # GDB-Skript für den Haupttest
47 |-- LaTeX                           # LaTeX-Dateien der Bachelorarbeit
48 |-- PCB                             # KiCad-Dateien für die erstellte PCB
49 |-- README.md                       # Projektbeschreibung für GitHub (Englisch)
50 |-- test_data                       # Daten für die durchgeführten Schreittests
51 |   |-- Writetest1                  # Versuch 1: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge
52 |   |-- Writetest2                  # Versuch 2: Manipulierte Schreibfunktion
53 |   |-- Writetest3                  # Versuch 3: Pseudo-randomisierte Daten
54 |   |-- `-- Writetest4              # Versuch 4: Daten bei Stromausfälle

```

2 Hardware-Design und Schnittstellen

2.1 Einleitung

Für diese Bachelorarbeit wurde eigens eine PCB in KiCad10 entwickelt. Die vollständigen Dateien, inklusive der Gerberdateien, liegen in dem [Projektverzeichnis auf Github](#).

Das Kernelement der Platine bildet der [STM32G431](#), ein 2019 veröffentlichter Mikrocontroller von STMicroelectronics[2]. Dieser ist preiswert und enthält alle wesentlichen Elemente, die für diese Arbeit benötigt wurden: 128 KB Flash, 32 KB SRAM, 3x SPI-Interfaces und außerdem frei konfigurierbares Direct Memory Access (DMA), das sogenannte DMAMUX. Bei den älteren STM32 waren die DMA-Kanäle festgelegt. Das hatte den Nachteil, dass man schon im Schaltplan sehr darauf achten musste, welche Pins für welche Peripherien verwendet werden, da ansonsten der DMA nicht verwendet werden konnte oder sich mit anderen Peripherien gedoppelt hat.

Weiterhin bietet der STM32G431 2x 12-bit ADCs, 2x 12-bit DACs und zahlreiche Timer, sodass potentielle Anwendungen wie das Einlesen von Sensordaten oder das Ausgeben von Audiodaten umgesetzt werden können.

Eine Übersicht der gesamten Features findet sich im [offiziellen Datenblatt](#), wobei sehr genau gelesen werden muss: Beispielsweise besitzt der Controller zwar 4 DAC Kanäle, aber nur 2 sind gleichzeitig nutzbar. ebenfalls kann eine ADC Genauigkeit von 16 Bit zwar erreicht werden, aber nur in einem speziellen Oversamplingmodus, wodurch die mögliche Abtastrate drastisch sinkt. Als Gehäuseform wurde LQFP48 gewählt, da dieses von Hand lötbar ist und genug Pins für alle benötigten Zwecke herausführt.

Es ist an dieser Stelle erwähnenswert, dass ST Microelectronics im höherwertigen Segment ebenfalls Mikrocontroller anbietet, die eine dezidierte SDMMC-Schnittstelle für den nativen SD-Modus von SD-Karten haben. Mit diesem lassen sich höhere Übertragungsraten erreichen, was für die vorliegende Arbeit aber nicht relevant war.

Die Platine wurde bei dem großen chinesischen Unternehmen [JLCPCB](#) bestellt. Die Bauteile wurden bei dessen Tochterunternehmen [LCSC](#) bestellt. Der Bill of Material (BOM) für die Bauteile befindet sich ebenfalls im Projektverzeichnis.

2.2 Idee und Schaltplan

Die Idee war der Entwurf einer Prototypplatine für einen STM32G431, die mit zwei MicroSD-Kartenslots ausgestattet wird. Darüber hinaus sollte die PCB so simpel wie möglich gehalten werden, da dies mein erster eigener Platinenentwurf ist und die nötige Erfahrung für komplexe Wechselwirkungen und Probleme fehlt.

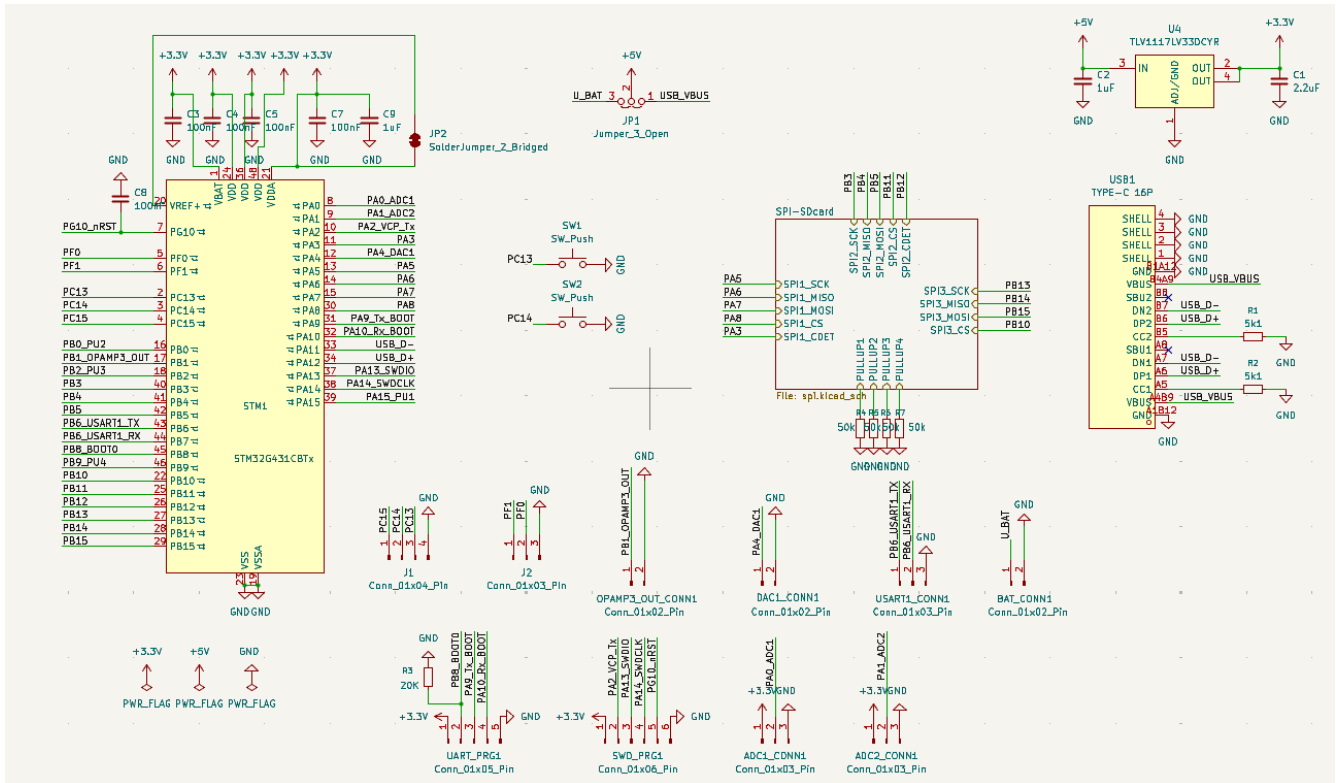


Abbildung 1: Hauptblatt des Schaltplans

Abbildung 1 zeigt den hauptsächlichen Schaltplan. In der Kurzschreibweise von Schaltungen werden oft die Leitungen für GND und VCC/VDD weggelassen. In ähnlichem Stile wurden in dem vorliegenden Schaltplan fast alle anderen Leitungen auch weggelassen und die Pins der Übersichtlichkeit halber mit Labels verbunden. Auch bei einem minimalen Design mussten einige Dinge für eine korrekte Funktionsweise beachtet werden, andere Features wurden hinzugefügt, um eine bessere Auswertung und Nutzung der Platine zu gewährleisten:

2.2.1 Spannungsversorgung

Die Spannungsversorgung soll hauptsächlich über ein 5V USB-C-Netzteil gewährleistet werden. Ein umsteckbarer Jumper ermöglicht allerdings auch die Versorgung über einen externen Akku, sofern er minimal 3,8 V bis maximal 6 V zur Verfügung stellt. Für die Spannungsregulation ist ein Texas Instruments TLV1117LV33DCYR Linearregler zuständig, da dieser außer zwei Kondensatoren keine komplexe, externe Beschaltung benötigt. Weiterhin wurden für eine stabile Spannungsversorgung alle VDD-Pins des STM32G431 mit einem 100 nF Kondensator ausgestattet.

2.2.2 USB-C

Die Platine ist mit einem 16-Pin USB Typ C Stecker ausgestattet. Da der Stecker beidseitig in die Buchse gesteckt werden kann, ist jeder Pin in zweifacher Ausführung vorhanden. Hauptsächlich ist der Stecker zur Spannungsversorgung gedacht gewesen. Damit die USB-Spannung von 5 V von einem Netzteil oder anderem Gerät überhaupt bereit gestellt werden kann, müssen die Pins CC1 und CC2 über einen 5,1 kΩ

Widerstand auf Masse gezogen werden. Es stehen mit USB-C auch höhere Spannungen zur Verfügung, wenn mit der Spannungsquelle über das USB Power Delivery Protokoll kommuniziert wird. Dies war für die vorliegende Arbeit aber nicht relevant.

Da STM32G431 aber einen USB 2.0 Controller besitzt, wurden ebenfalls die differenziellen Datenpins angeschlossen. Das erweitert die potentiellen Möglichkeiten der Platine für einen Datenaustausch mit dem Computer.

2.2.3 SPI-Interfaces

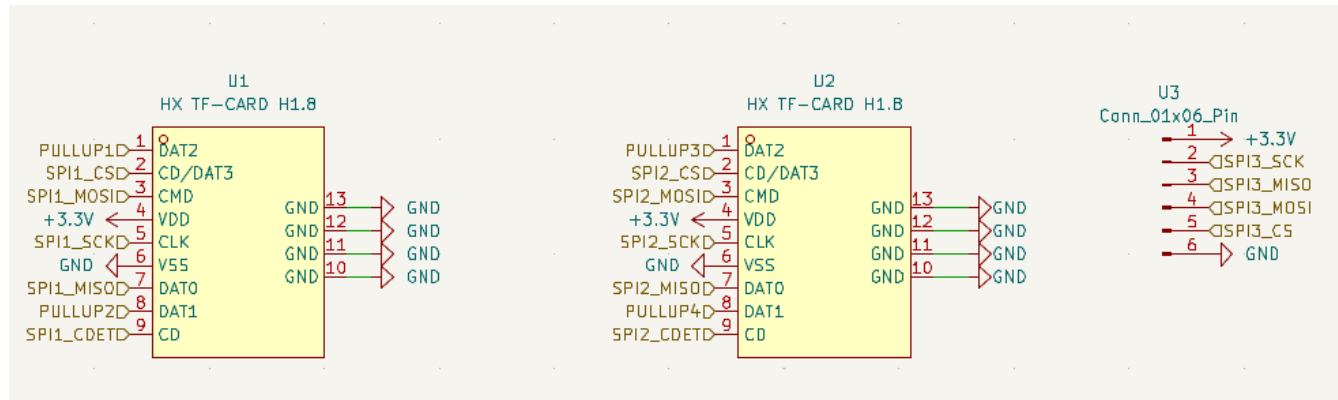


Abbildung 2: Schaltplan für die SPI-Interfaces

Die Hauptfunktionalität der Platine ist die Ansteuerung einer MicroSD Karte über das SPI-Protokoll. Der STM32G431 bietet hierfür eigene SPI-Hardwarecontroller, die als Master fungieren können und einen konfigurierbaren Takt vorgeben. Die Platine verbindet zwei der drei SPI-Peripherien mit MicroSD-Karten Steckplätzen und führt den dritten mit einer Pinleiste aus. Der gewöhnliche Kommunikationsmodus von SD-Karten, der SD-Modus, benötigt zwei weitere Pins. Diese werden per Pull-Up Widerstand auf 3,3 V Spannungspegel gebracht, um ein Übersprechen von den anderen Leitungen auf diese Pins zu unterbinden und damit potentielle Fehler zu minimieren. Die Pull-Up Widerstände sollten zunächst direkt über ungegenutzte STM32G431 Pins per Software konfiguriert werden. Beim Routing hat sich dies jedoch als schwierig herausgestellt, sodass physische Pull-Up Widerstände verwendet werden sollten. Durch die Änderungen in letzter Minute hat sich hierbei allerdings ein Fehler eingeschlichen und die Widerstände sind zu Pull-Down Widerständen geschaltet worden. Das dritte SPI-Interface wurde per Pinleiste rausgeführt, sodass die Flexibilität gewährleistet ist ein externes SPI-Gerät anzuschließen (wie z.B einen kleinen LCD-Bildschirm), falls dies gewünscht ist.

2.2.4 Programmier-Interfaces

Das Programmierinterface wurde in zwei Versionen herausgeführt:

Einerseits das UART-Programmierinterface über eine 5-Pin Leiste. Dieses erlaubt den STM32 über einen gewöhnlichen UART-Adapter zu programmieren. Der Nachteil ist hierbei allerdings, dass kein Debug-Interface vorhanden ist, sodass die Entwicklung eines Programms erschwert wird.

Andererseits wurden die Pins für das SWD-Interface mit einer 6-Pin Leiste herausgeführt. Dieses Interface benötigt einen dezidierten Programmieradapter. Allerdings lassen sich mit einigen Jumperumstellungen

auch die STM-Nucleo Boards dafür nutzen, da diese ebenfalls SWD intern für ihre Programmierung verwenden. ****HIER GENAUER SWD PROG ERKLAEREN MIT PINS UND BILDERN****

2.2.5 VREF und ADCs/DACs

Für die Referenzspannung für die ADC und DAC muss der Pin VREF mit VDDA verbunden werden und mit 1 μ F und 100 nF Kondensatoren abgesichert werden. Alternativ könnte man für eine Referenz den internen VREFBUF nutzen. In der vorliegenden Konfiguration besteht allerdings maximale Flexibilität. Um die Komplexität der Schaltung niedrig zu halten, wurden in diesem Platinendesign keine separaten Versorgungen für die digitale und analoge Spannung verwendet, sodass mit einem höheren Grundrauschen bei DAC und ADC gerechnet werden muss. Es wurden 2x ADCs und 2x DACs über 2-Pin Leisten herausgeführt. Einer der DAC ist intern mit dem OPAMP3 verschaltet und somit gepuffert.

2.2.6 Weitere Peripherien

Für potentielle Debugging und Kommunikationszwecke wurde ebenfalls per 3-Pin Leiste ein USART-Interface herausgeführt. Für eine einfache Eingabemöglichkeit wurden zwei Pins über einen Taster an GND angeschlossen. Diese sollen mit in Software zugeschalteten Pull-Up Widerständen verwendet werden. Weiterhin wurden 5 weitere ungenutzte Pins herausgeführt.

2.3 Routing und Design

2.3.1 Footprints

Da die Platine von Hand gelötet werden sollte, wurden alle Kondensatoren und Widerstände in der 0805 Größe gewählt. Alle Bauteile wurden schon in der Schaltplanphase darauf begutachtet, ob es sie in einer von Hand lötbaren Gehäuseform gibt.

Die Pads praktisch aller Bauteile wurden manuell etwas verlängert, um eine gute Handlötbarkeit zu gewährleisten. LCSC stellt für alle bei Ihnen verfügbaren Bauteile auch Footprints zur Verfügung. Die Footprints der spezielleren Bauteile, wie die des MicroSD-Kartenslots oder des TI Linearreglers, wurden mit dem Tool easyeda2kicad in einen KiCad Footprint überführt. Diese wiederum mussten teilweise auch leicht angepasst werden, weil z.B der Abstand der Pads zu den Bohrlöchern nicht die für die Produktion zulässigen Maße besessen hatte.

2.3.2 Design

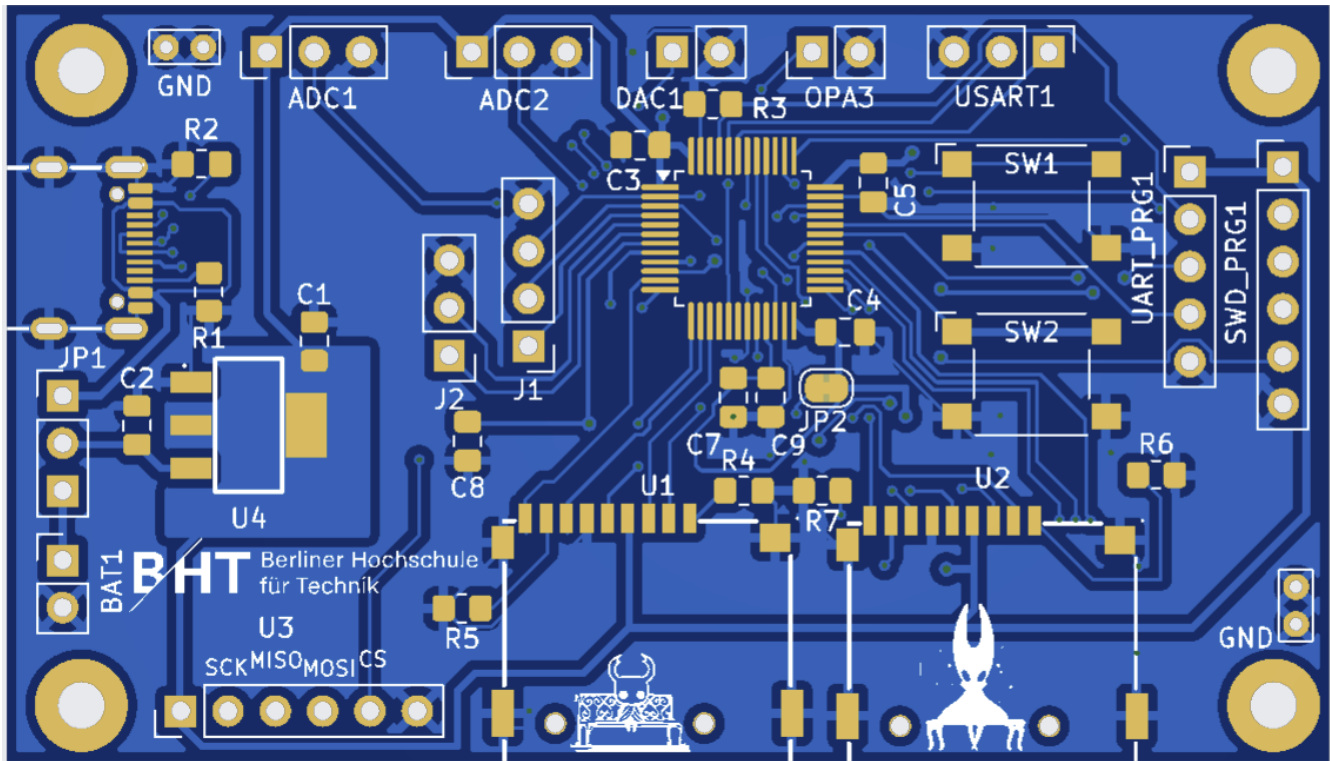


Abbildung 3: Vorschau der entwickelten PCB

Abbildung 3 zeigt die Vorderseite der Platine in dem Zustand, in dem sie bestellt wurde. Die Platine ist zweilagig angelegt und wird nur auf der Vorderseite per Hand bestückt.

Für Signalleitungen wurden 0,2 mm dicke Kupferbahnen, für die stromführenden Leitungen wurde eine Breite von 0,4 mm gewählt. Der grundlegende Abstand von Leiterbahn zu Leiterbahn beträgt 0,2 mm, während der Abstand zu nicht kontaktierten Bohrlöchern 0,25 mm beträgt. Dies sind überwiegend Vorgaben des Produzenten JLCPCB. Die Größe der Platine beträgt 71 mm x 40,5 mm.

Bei zukünftigen Projekten wird für die Beschaltung das echte physische Layout der Pins im Gehäuse stärker beachtet werden. So liegen beispielsweise die Pins PA8 bis PA12 am rechten oberen Rand des LQFP48-Gehäuses, während die Pins PA0 bis PA4 in der linken unteren Ecke liegen. Dies ist beim Schaltungslayout irrelevant, beim Routing der echten Leitungen hingegen führt dies zu überkreuzenden Leitungen. Daher sollte schon in der Schaltungsphase das spätere physische Layout mitbedacht werden.

2.3.3 Fertigung

Die bisherigen Maße und Überlegungen mussten schon konzeptionell und beim Entwurf im CAD-Programm beachtet werden. Für die Fertigung bei JLCPCB mussten weitere Optionen festgelegt werden: Der Materialtyp ist das übliche FR4 TG135 in der Farbe Blau. Die Löt pads werden nach dem Kupferätzprozess im "LeadFree HASL" Verfahren mit einer Schutzschicht Zinn überzogen. Auf diese Option muss besonders geachtet werden, da in Asien weiterhin bleihaltige Verzinnung üblich ist. Dies ist notwendig, da blankes Kupfer innerhalb kurzer Zeit oxidieren würde. Die Kupferdicke liegt bei den üblichen 35 µm.

Alle weiteren Möglichkeiten sind Spezialoptionen für Sonderanwendungen wie beispielsweise galvanisierte Kanten und sind für diese Platine nicht relevant.

2.3.4 Bestückung

Die Bestückung der Platine von Hand hat sich wie erwartet als große Herausforderung dargestellt. Vor allem das LQFP48 Gehäuse und der USB-C-Stecker konnten erst mit neu bestelltem NC-559 Flussmittel richtig gelötet werden. Stannol Flux X32-10i hingegen hat zu sehr vielen Brücken zwischen den Pins geführt. Weiterhin ist die Wahl der richtigen Lötspitze für die Drag-Soldering-Technik entscheidend. Entgegen der Intuition hat sich eine größere Lötspitze als besser herausgestellt, weil diese die Hitze, besonders in Kombination mit dem dickflüssigen Flussmittel, über eine größere Fläche gleichmäßiger verteilt.

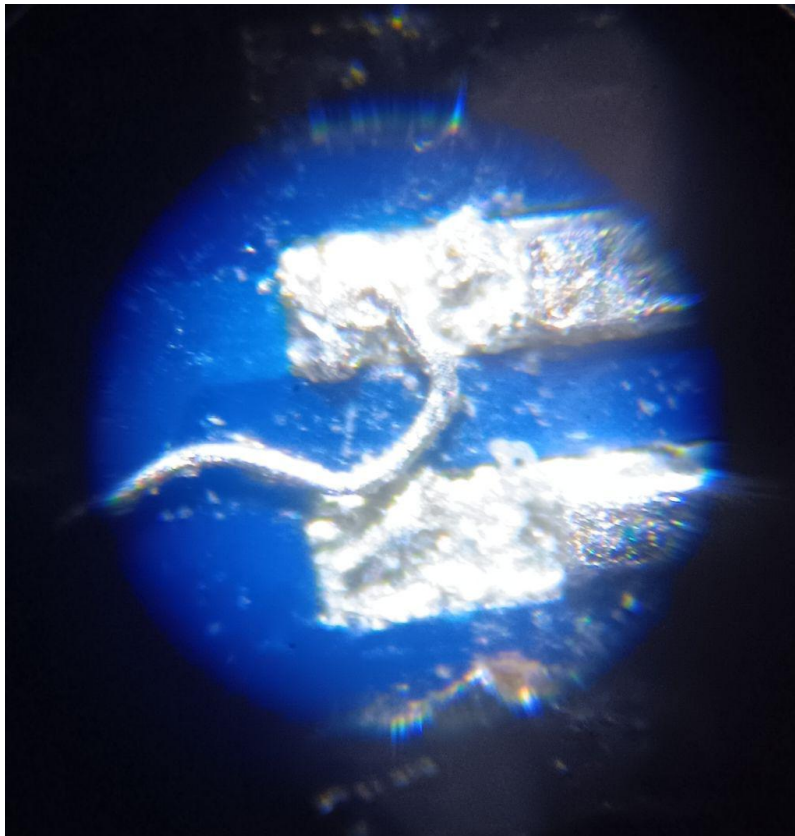


Abbildung 4: Lötbrücke bei einem der LQFP48-Pins des STM32G431

Abbildung 4 zeigt eine Mikroskopaufnahme einer ungewollten Brücke zwischen zwei Pins, die durch das Benutzen von Entlötlitze entstanden ist. Ohne ein Mikroskop ist dieses Drähtchen mit bloßem Auge nicht zu sehen.

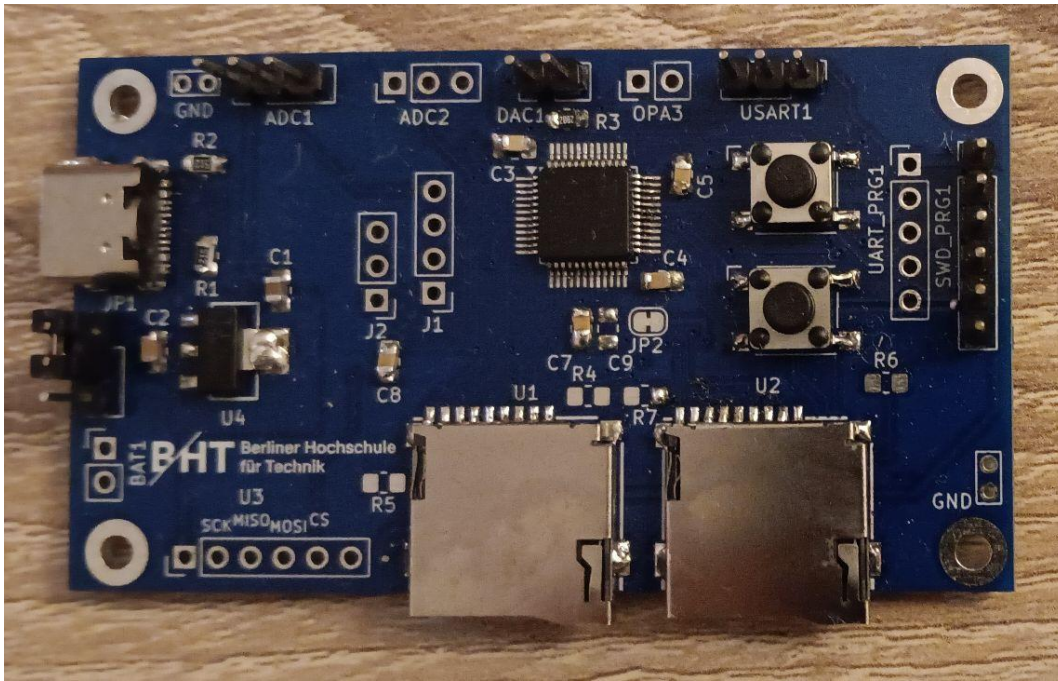


Abbildung 5: Platine in fast vollständiger Bestückung

Abbildung 5 zeigt eine der fertig gelöteten Platinen.

2.3.5 Inbetriebnahme und Probleme

Neben dem bereits erwähnten Problem der falsch beschalteten Pull-Up Widerstände wurden ebenfalls die Pins für die SPI2 und SPI3 Peripherien vertauscht. Dieser Fehler lässt sich durch eine Umkonfigurierung softwareseitig aber lösen.

Der mit Abstand unintuitivste und am schwierigsten auszumachende Fehler der Platine hängt allerdings mit der Spannungsversorgung für die SD-Kartenslots zusammen. Während den Testungen hat sich insbesondere der SD-Kartenslot, der an der SPI3-Peripherie angeschlossen ist, als besonders unzuverlässig herausgestellt. Dieser Fehler ist für den SPI3 SD-Kartenslot fatal und lässt sich ohne ein neues Design kaum beheben.

Abbildung 6 zeigt den Kanal MISO, also die Antwort der SD-Karte auf eine Initialisierungsanfrage. Die erwartete Antwort wäre 0xFF, also ein Signal, das durchgehend 3,3 V ist. Tatsächlich lässt sich aber ein kleine Kondensatorentladungskurve messen. Diese Entladungskurve war noch größer, als noch keine zusätzlichen 2,2 uF Kondensatoren provisorisch in der Nähe der SD-Kartenadapterversorgung gelötet wurden. Aber auch diese zusätzlichen, provisorischen Kondensatoren sind nicht in der Lage den Einbruch gänzlich wegzupuffern. Tatsächlich hängt der Spike auch vom verwendeten Netzteil ab, auch wenn sich an der Spannungsversorgung des Netzteils, sowie am TI Linearregler kein nennenswerter Einbruch messen lässt. Ohne einen 10 uF Kondensator ist der mit der SPI3 verbundene SD-Kartenslot daher leider nicht stabil benutzbar.

Die Peripherie an SPI1 hingegen macht weniger Probleme, da sie sich physisch näher an der Spannungsversorgung befindet und die Regelschleife des Linearreglers hier schnell genug die Versorgung nachregeln kann.

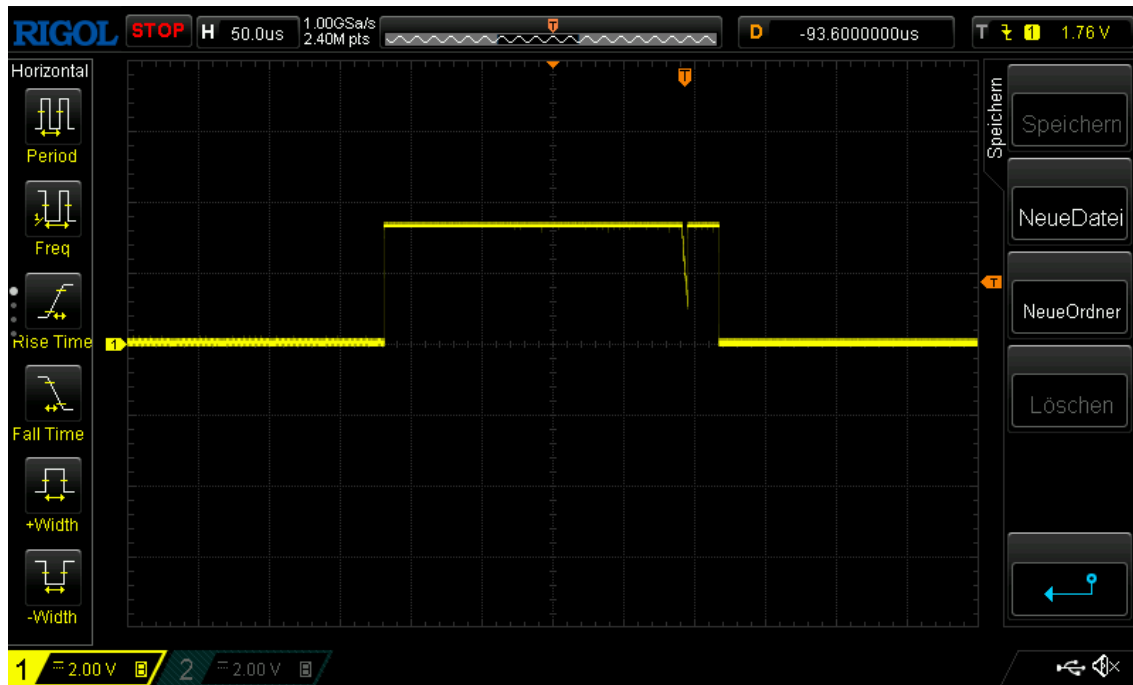


Abbildung 6: MISO: SD-Kartenantwort auf CMD0

Ein schwer aufzufindbarer, leicht lösbarer Fehler hing mit der Teilbestückung der Platine zusammen. Für erste Tests wurde die Platine nicht vollständig mit allem bestückt. So wurde der Widerstand R3 zunächst vergessen zu löten. *Abbildung 1* macht deutlich, dass dies allerdings ein Pull-Down Widerstand für den Boot-Pin PB8 ist. Wenn sich PB8 auf HIGH befindet, so lädt der STM32 in seinen internen Bootloader, um über die UART-Schnittstelle programmierbar zu sein. Wenn der Pin allerdings unverbunden keinen definierten Pegel erhält, so lädt er mal in den Bootloader und mal nicht, sodass sich die Platine in einem schwer diagnostizierbaren Zustand befindet.

2.4 Fazit

Alles in allem ist das Design für einen ersten Prototypen sehr gelungen und funktional. Der STM32 wird stabil mit Spannung versorgt und lässt sich über ein speziell konfiguriertes Nucleo-Board debuggen und programmieren. Die SPI1- und mit Einschränkungen auch die SPI3-Peripherien lassen sich für die weitere Arbeit verwenden. Eine große Lektion ist, dass schon beim Schaltungsdesign das spätere Layout mitbedacht werden muss, um vorrausschauene Entscheidungen bezüglich der Pinwahl treffen zu können.

SD-Karten scheinen durch ihren internen Aufbau phasenweise sehr kurzfristige Stromspitzen zu verursachen, die der verwendete LDO-Regler nicht ausregeln kann, wenn sich das Bauteil zu weit weg befindet. Daher ist es günstig diese in der Nähe der Spannungsversorgung zu platzieren und mit dezidierten Kondensatoren auszustatten.

Ein großes Ärgernis bei der Fehlersuche, insbesondere im Hinblick auf die SPI-Peripherie, die die Grundlage des SD-Kartentreibers bildet, ist eine fehlende Pinleiste für das Debugging. Ohne diese war es nur

mit größter Mühe und Präzision möglich einzelne SPI-Pins mit dem Oszilloskop zu messen. Ein Anschluss an einen Logikanalyser wäre hingegen gar nicht möglich gewesen.

All diese Punkte wären in einer zweiten Version der Platine aber leicht nachzubessern.

2.5 Testhardware

Bestellungen aus China benötigen mehrere Wochen Zeit.

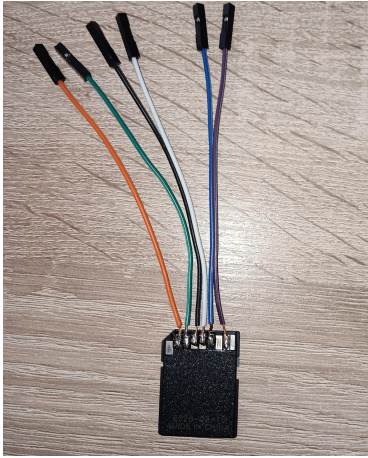


Abbildung 7: Erster gelöteter SD-Adapter

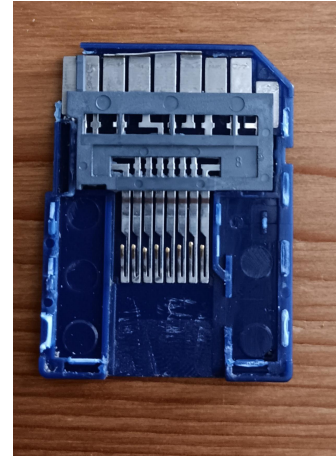


Abbildung 8: Aufgebrochener SD-Adapter

Um in der Zwischenzeit an der Arbeitsstellung weiterarbeiten zu können, wurde der in *Abbildung 7* gezeigte Adapter verwendet in Kombination mit einem ST Nucleo-G431RB. Diese Adapter finden sich meist bei jeder gekauften MicroSD-Karte und haben den Zweck die kleine Karte in ein größeres Format zu überführen. Wie so oft sind improvisierte Lösungen aber nicht sehr zuverlässig: Schon nach zwei Wochen musste ein neuer Adapter, diesmal mit gecrimpten Aderendhülsen, gelötet werden, da sich einzelne Stränge des Litzenkabels gelöst haben und zu Kurzschlüssen geführt haben.

3 SPI- und SD-Karten-Treiber

3.1 Embedded C++20

Für die Entwicklung des SPI- und SD-Karten-Treibers wurde bewusst auf modernes C++20 zurückgegriffen. Die Sprache erlaubt Hardware-Abstraktionen mit Templates und Berechnungen zur Compilezeit mit `constexpr` vollkommen ohne Laufzeitoverhead, sogenannte Zero-Cost-Abstractions. Moderne Sprachfeatures wie `enum class` und `namespace` bieten Typensicherheit und erhöhen die Lesbarkeit. Bedingte Kompilierung mit `if constexpr` wählt nur relevante Zweige zur Compilezeit aus und kompiliert den Rest erst gar nicht. Gleichzeitig besteht Abwärtskompatibilität zu C-Code, sodass beispielsweise die in C89 geschriebene FatFS-Bibliothek problemlos implementiert werden kann.

3.2 GPIO HAL

Um nicht ständig direkte und dadurch schwer lesbare Registerzugriffe innerhalb der Treiber vornehmen zu müssen, wurde eine kleine Zero-Cost-Abstraction für die Konfiguration und Ansteuerung der GPIOs

implementiert. Die in "GPIO_HAL.hpp" verwendete Template-Klasse `GpioPin` erhält die Basisadresse des verwendeten GPIO-Ports sowie die Pinnummer als Compile-Time-Parameter. Dadurch können konkrete GPIO-Pins typischer beschrieben und über eine einheitliche Schnittstelle konfiguriert werden. Die Klasse stellt unter anderem Funktionen zur Konfiguration als Ein- oder Ausgang, zur Auswahl von Pull-Up- bzw. Pull-Down-Widerständen sowie zur Konfiguration von Alternate Functions bereit. Da Port und Pin bereits zur Compile-Zeit feststehen und die bereitgestellten Funktionen als `static` implementiert sind, ist zur Laufzeit keine Objektinstanz erforderlich. Bei entsprechender Compileroptimierung können die Methoden daher auf die für den jeweiligen Pin notwendigen direkten Registerzugriffe reduziert werden. Die Abstraktion dient somit primär der Lesbarkeit und Wartbarkeit des Quellcodes, ohne einen relevanten Laufzeit- oder Speicher-Overhead einzuführen.

```
1  static void AF(uint8_t af)
2  {
3      // Alternate function mode
4      GPIOx->MODER &= ~(0b11 << (Pin * 2));
5      GPIOx->MODER |= (0b10 << (Pin * 2));
6
7      // AFRL or AFRH
8      if constexpr (Pin < 8)
9      {
10
11         GPIOx->AFR[0] &= ~(0xF << (Pin * 4));
12         GPIOx->AFR[0] |= (static_cast<uint32_t>(af) << (Pin * 4));
13     }
14     else
15     {
16         constexpr uint32_t shift = (Pin - 8) * 4;
17         GPIOx->AFR[1] &= ~(0xF << shift);
18         GPIOx->AFR[1] |= (static_cast<uint32_t>(af) << shift);
19     }
20 }
```

Quellcode 1: if constexpr bei der Alternate Function Implementation

Quellcode 1 zeigt beispielhaft den Code für die Implementierung der Alternate Function. Hier gestaltet sich `if constexpr` vorteilhaft, da die Pinnummer `Pin` bereits zur Compile-Zeit als Template-Parameter bekannt ist. Dadurch wird nur der tatsächlich benötigte Zweig für `AFR[0]` oder `AFR[1]` kompiliert, während der andere Zweig vollständig entfällt.

3.3 Debugging mit GDB

Ein absolut wesentlicher und sehr zeitintensiver Teil der Arbeit ist das Debugging der Hard- und Software. Zum Debugging wurde die VS Code Erweiterung [Cortex-Debug](#) verwendet. Diese nutzt im Hintergrund den [GNU Project Debugger GDB](#).

Mit der Erweiterung und der richtigen .svd Datei für den STM32G431 lassen sich direkt aus dem Editor Programme auf den STM32 flashen und Register und Variablen auslesen. Durch die Nutzung von CMake

lassen sich ausserdem bequem mehrere ausführbare Programme kompilieren und leicht wechseln.

So war recht schnell erreicht, dass das Projekt eine für das direkte Auslesen von Variablen unhandliche Größe angenommen hat. Für das Debugging wurde diese Arbeit inspiriert vom Rustprojekt [defmt](#), das Debuggingnachrichten direkt über die Debug Probe ST-Link ohne einen zusätzlichen UART-Kanal oder den SWO-Pin senden kann. Dies ist besonders nützlich, da der ST-Link zum Flashen der Programme ohnehin am Mikrocontroller angeschlossen sein muss.

Die minimale Lösung für diese Arbeit nutzt letztlich aus, dass GDB im angehaltenen Zustand immer den Speicher des Mikrocontrollers auslesen kann. Damit ist es möglich für Testprogramme GDB-Skripte zu entwerfen, die bestimmte Flags von Variablen oder ganze Variablen auswerten.

```
1  define spi_test
2  printf "\n\n SD Card Test:\n\n"
3
4  ###SPI1###
5  if sdtest.cur_spi == 1
6      printf "TESTING SPI1 PERIPHERAL \n"
7
8      #CARD DETECT
9      if sdtest.cardeetect == 1
10         printf "SPI1: CARD NOT DETECTED [FAILED]"
11     end
12     if sdtest.cardeetect == 0
13         printf "SPI1: CARD DETECTED [OK]"
14     end
15
16     #SD CARD INIT
17     if sdtest.initsd == SD_INIT_OK
18         printf "SPI1: SD CARD INIT [OK]\n"
19     end
20     if sdtest.initsd == 0x02
21         printf "SPI1: SD CARD INIT [FAILED: 0x02] => CMD0 FAILED\n"
22     end
23     if sdtest.initsd == 0x04
24         printf "SPI1: SD CARD INIT [FAILED: 0x04] => CMD8 FAILED\n"
25     end
26     if sdtest.initsd == 0x08
27         printf "SPI1: SD CARD INIT [FAILED: 0x08] => TIMEOUT: CMD55 + ACMD41 LOOP FAILED\n"
28     end
29     (...)
30 end
```

Quellcode 2: Auszug aus `spi_test.gdb`

Abbildung 2 zeigt einen Auszug aus dem erstellten GDB-Skript, welches die beiden SD-Kartenslots auf ihre Funktionsfähigkeit prüft.

```

1      SD Card Test:
2      TESTING SPI1 PERIPHERAL
3      SPI1: CARD DETECTED [OK]
4      SPI1: SD CARD INIT [OK]
5      SPI1: SD CARD WRITE SINGLE BLOCK [OK]
6      SPI1: SD CARD READ SINGLE BLOCK [OK]
7      SPI1: SD CARD READ MULTIPLE BLOCK [OK]
8      SPI1: SD CARD WRITE MULTIPLE BLOCK [OK]
9      SPI1: READ SINGLE BLOCK SAME AS WRITTEN [OK]
10     SPI1: READ MULTIPLE BLOCK SAME AS WRITTEN [OK]
11     SPI1: GOT CSD [OK]
12     SPI1: CARD SIZE IN MB: 31268
13     SPI1, FatFS: MOUNT [FAILED]
14     SPI1, FatFS: FOPEN [FAILED]
15     SPI1, FatFS: FWRITE [FAILED]
16     SPI1, FatFS: CLOSE [FAILED]
17     SPI1, littlefs: INIT [OK]
18     SPI1, littlefs: FORMAT [NONE]
19     SPI1, littlefs: MOUNT [OK]
20     SPI1, littlefs: OPEN [OK]
21     SPI1, littlefs: WROTE 32 BYTES
22     SPI1, littlefs: REWIND [OK]
23     SPI1, littlefs: READ 32 BYTES
24     SPI1, littlefs: READBUFFER IS SAME AS WRITEBUFFER [OK]
25     SPI1, littlefs: CLOSE [OK]
26     SPI1, littlefs: UNMOUNT [OK]

```

Quellcode 3: Debugausgabe von `spi_test.cpp` durch Anwendung von `spi_test.gdb`

Eine typische Ausgabe in der GDB-Debug Konsole zeigt *Quellcode 3*. Damit lässt sich mit einem Durchlauf des Programms feststellen, ob die zugrundeliegenden Funktionen des SPI/SD-Treibers alle ordnungsgemäß funktionieren und ob die Karte sich mit einem der beiden Dateisysteme mounten lässt und ob die grundlegenden Funktionen des Dateisystems, wie zum Beispiel das Lesen und Schreiben, fehlerfrei ausführbar sind.

GDB-Skripte sind ebenfalls in der Lage Breakpoints zu setzen. Mit dieser Funktion lässt sich ein einfaches Debug-Print in der GDB-Debug-Konsole realisieren.

```

1      break debug_gdb_print
2      commands
3          printf "%s", str
4          continue
5      end

```

Quellcode 4: Auszug aus `spi_test.gdb`

```

1 void debug_gdb_print(const char* str)
2 {
3     // breakpoint target only
4     //
5 }

```

Quellcode 5: Auszug aus `sd_test.cpp`

Quellcode 4 zeigt einen weiteren Auszug aus dem erstellten GDB-Skript `spi_test.gdb`. Dieser Code setzt einen Breakpoint bei der Funktion `debug_gdb_print` und gibt dessen `str`-Variable aus. Quellcode 5 zeigt den vollständigen Code der Funktion `debug_gdb_print`. Mit dieser Kombination ist es möglich die Funktion im Code aufzurufen. Diese macht codetechnisch nichts, veranlasst den Debugger aber kurz bei ihr anzuhalten, die Variable `str` ausgeben und sofort den Programmablauf fortzuführen. Während diese Funktionalität nicht in zeitkritischen Schleifen zum Einsatz kommen kann, ist sie für allgemeine Debuggingnachrichten sehr gut anwendbar. Gleichzeitig ist kein zusätzliches, externes Modul notwendig, sondern es nutzt die bereits vorhandenen Funktionalitäten des GDB-Debuggers aus.

3.4 SPI

3.4.1 Anschlussbelegung

Tabelle 1: Anschlussbelegung der Standard SD-Karte am STM32G431 im SPI-Modus für SPI1

Pin	SD-Modus	SPI-Modus	Beschreibung (SPI)	STM32 G431 Pin
1	DAT3 / CD	CS	Chip Select (Low-aktiv)	GPIO (z.B. PA8)
2	CMD	DI	Data In (Dateneingang)	PA7 (MOSI)
3	VSS1	VSS	Masse	GND
4	VDD	VDD	Stromversorgung (3,3 V)	3,3 V
5	CLK	SCLK	Taktleitung	PA5 (SCK)
6	VSS2	VSS	Masse	GND
7	DAT0	DO	Data Out (Datenausgang)	PA6 (MISO)
8	DAT1	<i>Nicht genutzt</i>	Pull-Up/nicht belegt	–
9	DAT2	<i>Nicht genutzt</i>	Pull-Up/nicht belegt	–

Tabelle 1 verdeutlicht die Anschlussbelegungen, die überwiegend für die Testhardware am Nucleo-G431RB verwendet wurden.

Abbildung 8 verdeutlicht, dass SD-Adapter lediglich die Leitungen der MicroSD-Karte umlenken. Die neue Belegung für SPI1, mit der die Platine arbeitet, findet sich in Tabelle 2.

Tabelle 2: Anschlussbelegung der MicroSD-Karte am STM32G431 im SPI-Modus für SPI1

MicroSD-Pin	Signalname	Verbindung am STM32G431 / Platine
Pin 1	DAT2	3.3V über internen Pull-up
Pin 2	CS	GPIO (z.B. PA8)
Pin 3	DI (MOSI)	PA7
Pin 4	VDD	3.3V Hauptversorgung
Pin 5	SCK	PA5
Pin 6	VSS	GND
Pin 7	DO (MISO)	PA6
Pin 8	DAT1	3.3V über internen Pull-up

Abbildung 9 zeigt die Nummerierungsweise bei den beiden Größen von SD-Karten.

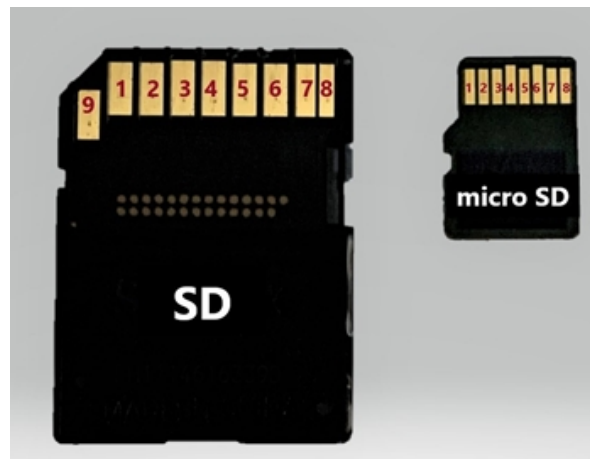
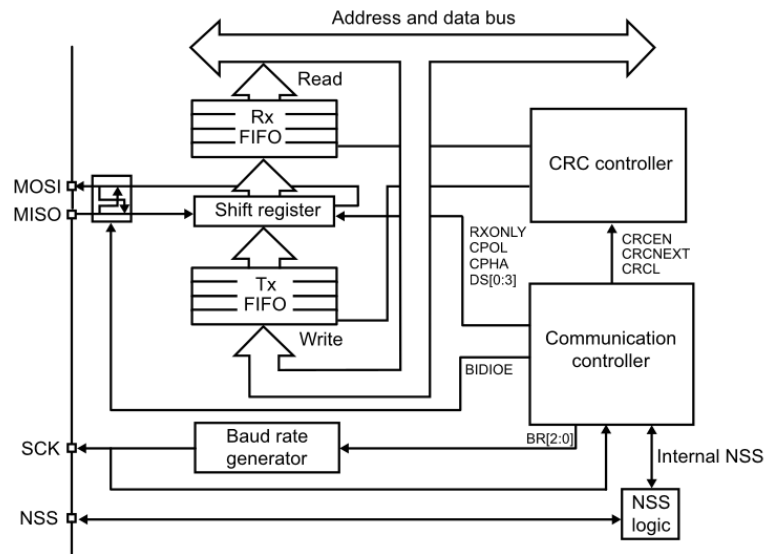


Abbildung 9: Pinnummerierung SD-Karten[14]

3.4.2 SPI-Peripherie des STM32G431



MS30117V1

Abbildung 10: Blockdiagramm für die SPI-Peripherie des STM32G431[4]

Abbildung 10 zeigt das Blockdiagramm für die SPI-Peripherie der STM32G4-Serie. Der relevanteste Teil ist das Shiftregister. Das oberste Bit des Shift-Registers wird auf MOSI ausgegeben. Gleichzeitig wird das Bit auf MISO eingelesen. Es liegt in der verwendeten Konfiguration immer eine Full-Duplex Übertragung vor. Um Daten über MISO zu empfangen, muss der Master eine Übertragung initiieren und entsprechend Taktzyklen erzeugen. Dabei werden gleichzeitig Daten über MOSI übertragen. Aus dem Rx-FIFO können Daten gelesen werden, sobald das RXP-Flag gesetzt ist. Dieses Flag wird gesetzt, wenn so viele Bits wie im Feld "Data size" des SPIx_CR2 Registers spezifiziert mit Daten gefüllt wurden.

Der Baudratencontroller ist für die SCLK und somit für die Übertragungsgeschwindigkeit zuständig. Er nutzt den APB-Bus-Takt und teilt ihn, falls im BR-Register konfiguriert, durch einen Prescaler.

Für den Zweck der SD-Karten-Kommunikation wurde die SPI-Peripherie als Master konfiguriert mit einem frei wählbaren, softwareseitig zuschaltbaren NSS-Pin (chip select).

```

1  template<uintptr_t PortBase, uint32_t Pin>
2  struct ChipSelect {
3  private:
4      static inline GPIO_TypeDef* const port = reinterpret_cast<GPIO_TypeDef*>(PortBase);
5  public:
6      ChipSelect() {
7          port->BSRR = (1U << (Pin + 16));
8      }
9
10     ~ChipSelect() {
11         port->BSRR = (1U << Pin);
12     }
13 };

```

Quellcode 6: Chip select als Klasse

Der Quellcode 6 zeigt die Stärke von C++ im Gegensatz zu C in diesem Kontext. Der NSS-Pin muss am Anfang der meisten SD-Funktionen wie `SD_ReadBlock(...)` oder `SD_SendCmd(...)` auf Low konfiguriert werden und beim Verlassen der Funktion wieder auf High. Mit `ChipSelect<GPIOA_BASE, 8> chipselect_tmp;` würde ein Objekt erzeugt werden. Beim Konstruieren, also dem Erzeugen des Objektes, würde der Konstruktor Pin 8 von GPIOA auf Low setzen und beim Verlassen der Funktion würde der Destruktor aufgerufen werden und den Pin wieder auf High setzen. In C müssten manuell alle Stellen gefunden werden, bei dem die Funktion verlassen wird oder Laufzeitkosten in Kauf genommen werden mit dem Aufruf einer Helferfunktion. Die Informationen welcher GPIO-Port mit welchem Pin benutzt werden soll steht zur Compilezeit fest, der verwendete Cast ist `static inline const`, sodass ein guter C++-Compiler die gesamte Klasse vollständig auf zwei direkte Registerzugriffe reduzieren kann.

3.4.3 SPI Treiber

”spidriver.hpp” kapselt die hardwarenahe Konfiguration und Kommunikation über die SPI-Peripherie des STM32G431. Die Initialisierung konfiguriert den jeweiligen SPI-Controller als Master mit softwaregesteuertem NSS und einer Datenbreite von acht Bit. Die Methode `static uint8_t Transfer(uint8_t data)` kapselt schließlich das Senden und Empfangen eines Bytes und übernimmt dabei die notwendige Synchronisation mit den Statusflags der SPI-Peripherie. Zusätzlich kann der verwendete Baudratenteiler über die Methode `static void SetHighSpeed(SpiDiv div)` verwendet werden, um zwischen einer Initialisierung mit niedriger und einer späteren Kommunikation mit höherer Übertragungsgeschwindigkeit zu wechseln.

3.5 Initialisierung der SD-Karte in den SPI-Modus

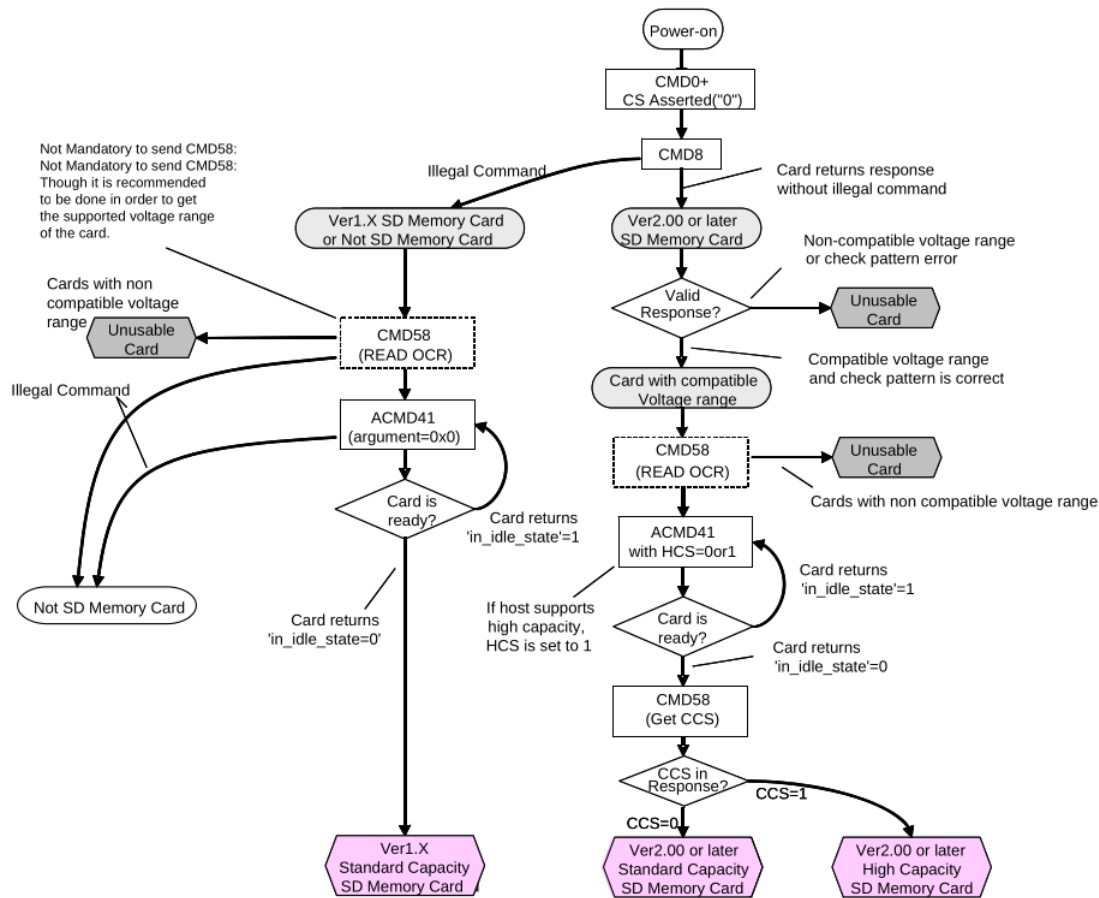


Abbildung 11: Initialisierung der SD-Karte in den SPI-Modus[6]

Abbildung 11 zeigt den vollständigen Initialisierungsprozess als Flussdiagramm aus der SD-Spezifikation. Die Initialisierung und Treiberfunktionen finden sich in der Datei `sdcardsdriver.hpp`. Die Initialisierung ist etwas verkürzt, wie auf der Abbildung zu sehen ist CMD58 nur notwendig, wenn Interesse an den unterstützten Spannungsbereichen der SD-Karte besteht. 3,3 V Betriebsspannung ist aber für praktisch jede auf dem Markt gebräuchliche SD-Karte zulässig. Zumal die Spezifikation den Nutzer hier vor ein klares Henne-Ei-Problem stellt. Ohne eine geeignete Spannungsversorgung lässt sich der unterstützte Spannungsversorgungsbereich der Karte nicht auslesen.

Weiterhin unterschlägt die Abbildung die Tatsache, dass für die Benutzung von ACMD41 zunächst einmal CMD55 gesendet werden muss. Diese Tatsache muss aus einem anderen Teil der Dokumentation entnommen werden.

```

1  template<typename Config>
2  uint8_t SD_InitSPI()
3  {
4  (...)
5  ChipSelect<Config::PortBase, Config::Pin> chipselect_tmp;
6  // CMD0
7  //Give it some tries, does not work the first time, because card is busy...
8  for (int i = 0; i < 20; ++i)
9  {
10     cmd0_resp = SD_SendCmd<Config>(0, 0, 0x95, nullptr, 0);
11
12     if (cmd0_resp == 0x01)
13         break;
14 }
15
16 if (cmd0_resp != 0x01)
17     return SD_INIT_ERROR_CMD0;
18
19 // CMD8
20 SD_SendCmd<Config>(8, 0x000001AA, 0x87, cmd8_resp, 4);
21
22 if (cmd8_resp[0] != 0x01 || cmd8_resp[4] != 0xAA)
23     return SD_INIT_ERROR_CMD8;
24 (...)
25 }

```

Quellcode 7: Auszug aus der Funktion SD_InitSPI()

Quellcode 7 zeigt einen Auszug aus der Initialisierungsfunktion, Quellcode 8 beschreibt die verwendete SD_SendCmd Funktion. SD_SendCmd nutzt den zugrundeliegenden SPI-Treiber, um auf der im Templateparameter Config spezifizierten Peripherie die korrekten Daten über die bereits implementierte Transfer(uint8_t Data) Funktion zu senden. Die Daten für die Peripherien können in der Datei init_conf.hpp konfiguriert werden. Für die vorliegende Arbeit und Platine sind diese natürlich fix, allerdings lässt der STM32G431 einiges an Spielraum zu was variable Peripheriekonfiguration angeht.

Wichtig ist hierbei, dass bei CMD0 und CMD8 zunächst korrekte CRC-Werte für ihre Befehle benötigen. Das SD-Protokoll spezifiziert hier für den SPI-Modus etwas anderes als für den SD-Modus: Im normalen SD-Modus wird bei grundsätzlich jedem CMD die Prüfsumme geprüft und muss daher mitgesendet werden. Im SPI-Modus ist diese Prüfsummenberechnung grundsätzlich deaktiviert und kann optional mit CMD59 aktiviert werden. Dies gilt aber nicht für CMD0 und CMD8, diese Befehle brauchen immer einen korrekten CRC7-Wert.

```

1  template<typename Config>
2  uint8_t SD_SendCmd(uint8_t cmd, uint32_t arg, uint8_t crc, uint8_t *response_buf, uint8_t
   ↪  extra_bytes)
3  {
4
5      uint8_t r1;
6      uint16_t timeout = 0;
7
8      // send cmd
9      SpiDriver<Config::SpiBase>::Transfer(cmd | 0x40);
10
11     // 4 byte argument
12     SpiDriver<Config::SpiBase>::Transfer((uint8_t)(arg >> 24));
13     SpiDriver<Config::SpiBase>::Transfer((uint8_t)(arg >> 16));
14     SpiDriver<Config::SpiBase>::Transfer((uint8_t)(arg >> 8));
15     SpiDriver<Config::SpiBase>::Transfer((uint8_t)arg);
16
17     // 1 byte CRC (SPI-Mode ignores it, but still needed even then... Needed for CMD0 and CMD8 though!)
18     SpiDriver<Config::SpiBase>::Transfer(crc);
19     (...)
20 }

```

Quellcode 8: Auszug aus der Funktion SD_SendCmd()

Quellcode 8 verdeutlicht wie das Senden von einzelnen SD-Befehlen über die SPI-Peripherie prinzipiell funktioniert. Tatsächlich wurde die Funktion SD_SendCmd selten verwendet, da nach dem Senden des 4-Byte Arguments meistens noch für die konkreten SD-Befehle spezifische Abläufe folgen.

3.6 Lese- und Schreiboperationen

3.6.1 Begriffsklärung Sektor, Block, Cluster, Page

Für die weitere Arbeit müssen hier im Zusammenhang mit Blockspeicher zunächst häufig verwendete Begriffe geklärt werden: Ein Sektor bezeichnet die kleinste physikalisch adressierbare Speichereinheit eines Datenträgers. Ein Block bezeichnet dagegen die kleinste logische Speichereinheit, die von einem Betriebssystem oder Dateisystem adressiert wird. Ein Block kann also größer sein als ein Sektor, aber nicht anders herum.

Ein Cluster ist genau dasselbe wie ein Block, taucht aber im Zusammenhang mit dem FAT-Dateisystem auf, wohingegen der Block in der Unixwelt geläufiger ist.

Im Zusammenhang von Flashspeicher wird die Page als kleinste programmierbare Einheit beschrieben, wohingegen der Erase Block die kleinste löschbare Einheit beschreibt.

Leider werden in Spezifikationen und Literatur die Begrifflichkeiten nicht eindeutig verwendet. So spricht man im Zusammenhang von SD-Karten in aller Regel von Blöcken, die beschrieben werden, obwohl der Begriff des Sektors hier manchmal synonym verwendet wird. Hier wird im Folgenden von Blöcken die Rede sein, womit aber die kleinste von der SD-Karte adressierbare Einheit gemeint ist.

3.6.2 SD-Karte als Blockspeicher

Die essenzielle Funktionalität der SD-Karte ist das Lesen und Schreiben von Blöcken. Für die verwendeten Dateisysteme ist die SD-Karte als Blockspeicher abstrahiert worden. Das Dateisystem „weiß“ gar nicht, dass es eine SD-Karte verwendet, sondern ihm werden lediglich Funktionen zur Verfügung gestellt, mit denen es einzelne oder mehrere Blöcke schreiben und lesen kann.

Die Funktionen des SD-Treibers finden sich in der Datei `src/sdcarddriver.hpp`.

3.6.3 Lesefunktionen

Für das blockweise Lesen eines Blocks wurde die Funktion `uint8_t SD_ReadBlock(uint32_t block_addr, uint8_t *buffer)` implementiert. Die Funktion `uint8_t SD_ReadBlocks(uint32_t block_addr, uint32_t block_count, uint8_t *buffer)` hingegen implementiert das Lesen von mehreren Blöcken. Das Argument `uint32_t block_count` gibt dabei an wie viele Blöcke hintereinander gelesen werden sollen, wohingegen `uint32_t block_addr` den Startblock angibt, angefangen bei Block 0. Da das Feld 32-bit groß ist, lassen sich damit genau $\frac{2^{32} \cdot 512}{1024^4} = 2 \text{ TB}$ an Daten adressieren. Dies gilt für den modernen SDHC/SDXC-Kartenstandard und somit für den ansoluten Größteil aller heutigen SD-Karten. Der ältere SDSC-Standard hat auf eine Byteadressierung gesetzt und hat somit ein Speicherlimit von 2 GB. Vereinzelt braucht es allerdings SDSC-Karten noch für alte Hardware aus den frühen 2000er Jahren, die den neuen Standard nicht implementiert hat.

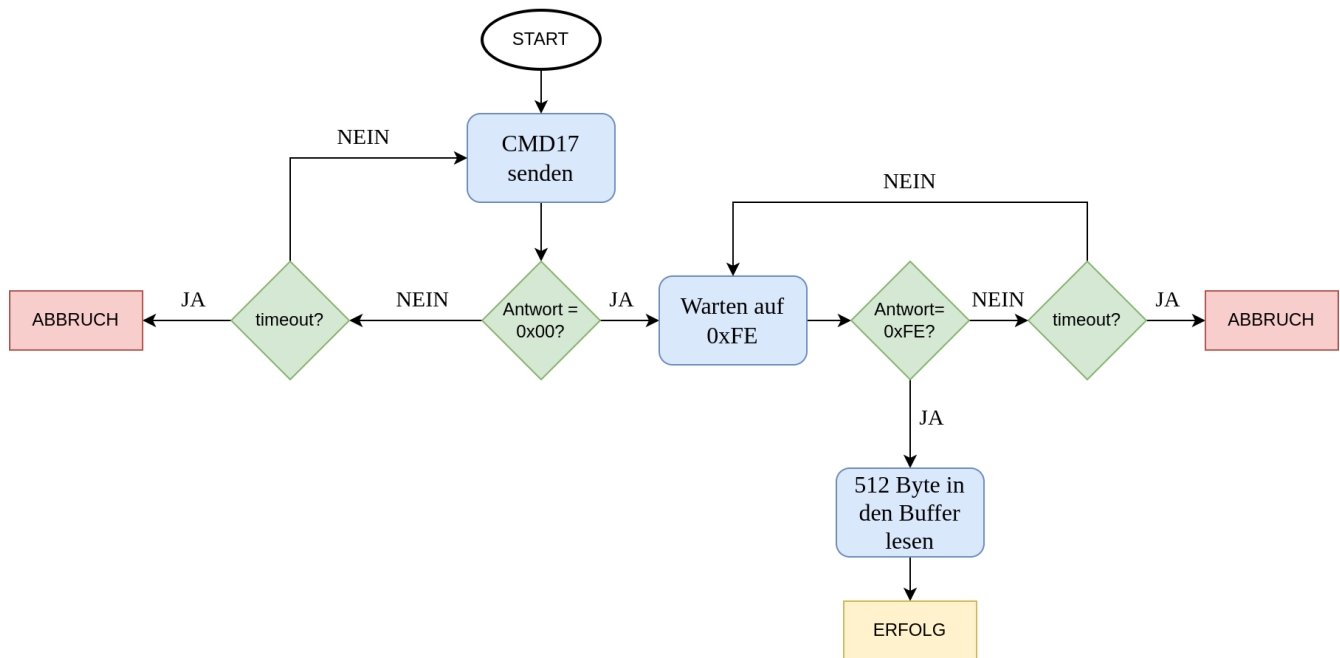


Abbildung 12: Flussdiagramm des prinzipiellen Ablaufs der `SD_ReadBlock`-Funktion

Abbildung 12 zeigt ein Flussdiagramm für die `SD_ReadBlock`-Funktion. CMD17 entspricht dem Befehl „Read Single Block“ aus der SD-Spezifikation und kann nach der Initialisierung der SD-Karte in den SPI-Modus gesendet werden. Wenn die Karte nach dem Empfang des CMD17 bereit ist, so sendet sie ein 0x00-Byte zurück und im Anschluss ein 0xFE-Byte. Nach dem 0xFE-Byte folgt dann der angeforderte Block bestehend aus 512-Bytes an Daten, die in einen Buffer eingelesen werden können.

An zahlreichen Stellen wird durch einen Timeout sichergestellt, dass die Funktion bei Fehlschlag nicht den gesamten Programmfluss blockiert. Die SD-Spezifikation gibt hier keine Vorgaben, also wurden Werte bis zur Bewährung getestet.

```
1 // wait for start 0xFE
2 start_time = TIM2->CNT;
3 do {
4     r1 = SpiDriver<Config::SpiBase>::Transfer(0xFF);
5 } while ((r1 != 0xFE) && ((TIM2->CNT - start_time) < T_OUT2_US));
6
7 if (r1 != 0xFE) {
8     //SD_CS_DESELECT();
9     return 0x02; // timeout-error waiting for 0xFE
10 }
11
12 // read 512 byte blockdata
13 for (uint16_t i = 0; i < 512; i++) {
14     buffer[i] = SpiDriver<Config::SpiBase>::Transfer(0xFF);
15 }
16
```

Quellcode 9: Auszug aus der Funktion SD_ReadBlock

Quellcode 9 zeigt beispielhaft einen Auszug der Funktion SD_ReadBlock, der auf das 0xFE-Byte wartet und danach 512-Bytes empfängt. Da die SPI-Peripherie immer gleichzeitig sendet und empfängt, müssen für den Empfang von Bytes willkürliche Daten gesendet werden. Ebenfalls zeigt der Auszug wie mittels eines Hardwaretimers der Timeout implementiert wurde.

Die Funktion SD_ReadBlocks ist die etwas komplexere Version, die für das Lesen von multiplen Blöcken genutzt werden kann. Dies hat den Vorteil, dass es den Overhead im Vergleich mit der Payload minimiert: Während bei der SD_ReadBlock-Funktion für alle 512 Bytes einmal CMD17 gesendet werden muss zuzüglich aller potentiellen Wartezeiten im SD-Controller, so reicht ein CMD18-Befehl für beliebig viele 512-Byte Blöcke.

Prinzipiell ist der Programmablauf ähnlich: Zunächst muss CMD18 gesendet werden. Das entspricht dem Befehl „Read Multiple Block“ aus der Spezifikation. Nach dem 0xFE-Byte folgen aber so lange Bytes, bis ein CMD12 („Stop Transmission“ den Datenfluss unterbricht.

3.6.4 Schreibfunktionen

```
uint8_t SD_WriteBlock(uint32_t block_addr, const uint8_t *buffer) uint8_t SD_WriteBlocks(uint32_t
block_addr, uint32_t block_count, const uint8_t *buffer)
```

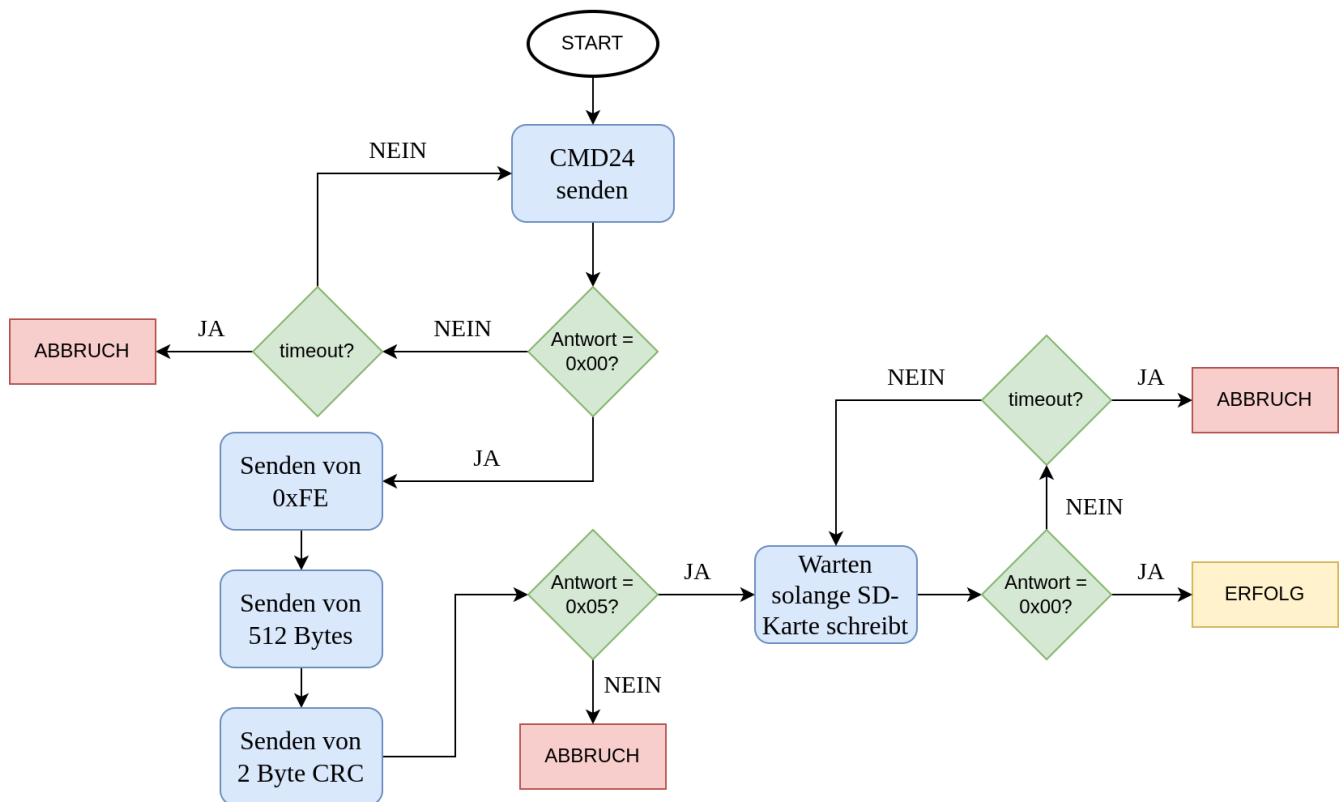


Abbildung 13: Flussdiagramm des prinzipiellen Ablaufs der `SD_WriteBlock`-Funktion

```

1  BSP
2  }

```

Quellcode 10: Auszug aus der Funktion `SD_SendCmd()`

3.6.5 Statusfunktionen

```

uint8_t SD_GetCSD(uint8_t *buffer, const uint8_t BUF_SIZE) uint8_t parse_csd(const uint8_t*
rawcsd, Csd_Common* csd)

```

```

1  BSP
2  }

```

Quellcode 11: Auszug aus der Funktion `SD_SendCmd()`

4 Dateisysteme

4.1 Einleitung

Eine umfassende Einführung in die Grundlagen von Dateisystemen wird von Tanenbaum und Bos in ihrem Standardwerk *Modern Operating Systems* bereitgestellt [1, S. 263 ff.]. Laut den Autoren werden

drei essenzielle Anforderungen an die Langzeitspeicherung von Daten gestellt:

1. Es muss möglich sein, eine sehr große Menge an Informationen zu speichern.
2. Die Informationen müssen das Beenden des Prozesses, der sie verwendet, überdauern.
3. Mehrere Prozesse müssen gleichzeitig auf die Informationen zugreifen können.

Tanenbaum und Bos nutzen in ihrem Buch den Begriff des Prozesses zur Ressourcenverwaltung und -abstraktion, während diese Notwendigkeit in der vorliegenden Arbeit entfällt, da das Programm direkte Kontrolle über die gesamte Hardware des Mikrocontrollers besitzt. Im Prinzip könnte das Problem eine große Menge an Information zu speichern bereits mit zwei Operationen lösen:

1. Einen wählbaren Block lesen
2. Einen wählbaren Block schreiben

Das sind exakt die Operationen, die in dem vorherigen Kapitel 3 für die SD-Karte implementiert wurden. In der Praxis sind diese zwei Operationen allerdings zu primitiv, um Daten effektiv zu verwalten. Tanenbaum und Bos stellen beispielhaft folgende Fragen:

- Wie findet man eine bestimmte Information
- Wie verhindert man (auf Mehrbenutzersystemen), dass ein Benutzer nicht die Daten eines anderen ausliest?
- Woher weiß man, welche Blöcke frei sind?

Für diese und viele andere praktische Probleme hilft das Konzept der Datei. Die Datei ist etymologisch ein Neologismus aus Daten und Kartei. Im Kontext moderner Betriebssysteme definieren Tanenbaum und Bos[1] Dateien als logische Einheiten von Information, die von Prozessen erzeugt werden und diese überdauern. Eine Datei soll erst gelöscht werden, wenn ein Besitzer dies explizit veranlasst.

Wie Dateien strukturiert, benannt, aufgerufen, genutzt, geschützt, implementiert und verwaltet werden ist Sache des Dateisystems. Ein Dateisystem löst (je nach Funktionsumfang) viele der oben gestellten Anforderungen. Aus Sicht des Nutzers ist der wichtigste Aspekt eines Dateisystems, wie es sich darstellt: Was eine Datei ausmacht, wie Dateien benannt und geschützt werden, welche Operationen auf Dateien erlaubt sind. Die Details darüber, ob verkettete Listen oder Bitmaps verwendet werden, um den freien Speicherplatz zu verwalten, oder wie viele Sektoren ein logischer Festplattenblock enthält, sind für den Nutzer von keinem Interesse, obwohl sie für die Entwickler des Dateisystems von großer Bedeutung sind[1]. Zusammenfassend lässt sich sagen, dass ein Dateisystem eine für den Anwendungszweck geeignete Struktur in einen rohen Speicherbereich bringt. Während Tanenbaum und Bos den Schwerpunkt auf Dateisystemen für Betriebssysteme legen, was sicherlich der übliche Anwendungsbereich ist, geht es in dieser Arbeit um die Verwendung von Dateisystemen in eingebetteten Systemen, bzw. konkret auf einem STM32-G431 und deren Vor- und Nachteile.

4.2 Layout eines Dateisystems

Während das Design eines Dateisystems grundsätzlich frei entschieden werden kann, folgt die Struktur meist einem ähnlichen Muster, sodass die Klärung einiger üblicher Begriffe nötig ist.

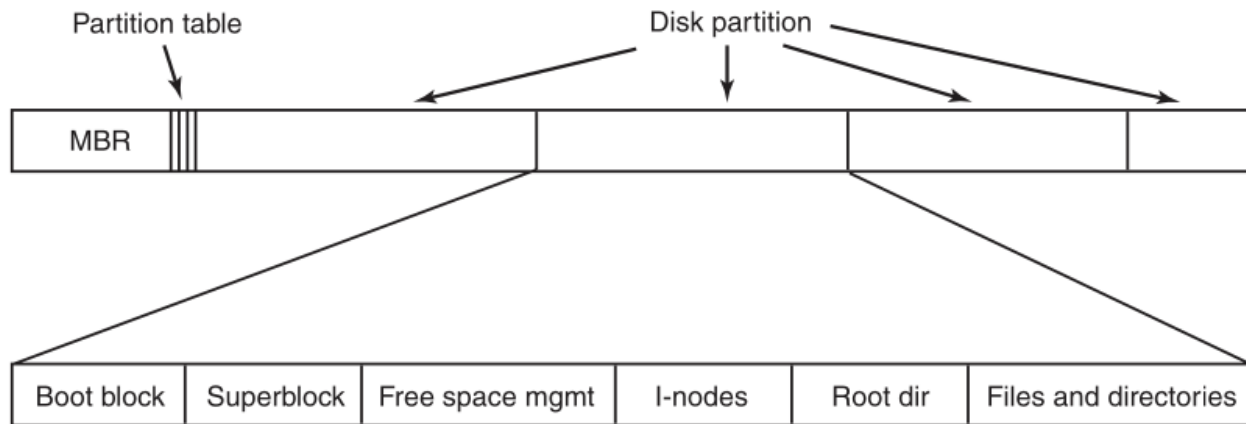


Abbildung 14: Ein mögliches Dateisystemlayout [1, S. 282]

Abbildung 14 zeigt ein mögliches Layout eines Dateisystems nach Tanenbaum und Bos [1, S. 282]. Der erste Sektor eines Speichermediums wird reserviert für den MBR (Master Boot Record) und wird üblicherweise zum Starten eines Computers verwendet. Das Ende des MBR enthält die Partitionstabelle, in welcher eine der Partitionen als aktiv markiert ist. Ein physisches Medium kann mehrere Partitionen und somit mehrere Dateisysteme beinhalten. Wenn der Computer gebootet wird, dann stellt ein Programm fest, welche Partition aktiv ist und führt die Anweisungen in dem "Boot block" aus. Üblicherweise folgt darauf Superblock, in dem wichtige Informationen über das Dateisystem stehen wie der Typ des Dateisystems oder die Anzahl der vorhandenen Blöcke. Im Anschluss folgen typischerweise Informationen über die freien Blöcke des Dateisystems, die beispielsweise in Form einer Zeigerliste organisiert sind. Darauf folgen die sogenannten I-Nodes, ein Array von Datenstrukturen, das für jede Datei existiert und alle relevanten Datei-Metadaten enthält. Danach schließt sich das Wurzelverzeichnis (Root Directory) an, welches die Wurzel der gesamten Dateisystemstruktur bildet. Der verbleibende Speicherplatz des Datenträgers steht schließlich für alle weiteren Verzeichnisse und die eigentlichen Dateien zur Verfügung[1, S. 282].

Im folgenden Abschnitt soll es um eben diesen verbleibenden Speicherplatz und dessen mögliche Organisation gehen.

4.3 Design eines Dateisystems

Die Autoren des littlefs-Dateisystems kategorisieren auf ihrer [GitHub Seite](#) Dateisysteme sehr übersichtlich in vier Gruppen[15]:

4.3.1 Nicht-resiliente, blockbasierte Dateisysteme

Bekannte Vertreter sind hier FAT für Microsoftsysteme und ext2 für Linux-Betriebssysteme. Diese grundlegenden Systeme führen Aktualisierungen direkt an Ort und Stelle und ohne Absicherungsmechanismen

durch. Dadurch sind sie bei plötzlichen Stromausfällen sehr anfällig für Datenkorruption. Im Zusammenhang mit Flashspeicher ist ein weiterer Nachteil, dass derselbe Block somit mehrfach beschrieben wird und es zur einer schnellen Abnutzung des Speichers kommt. Vorteile sind jedoch, dass diese Dateisysteme meistens sehr schnell und klein sind, sowie einfach in der Implementation.

4.3.2 Logging-Dateisysteme

Reine Logging-Dateisysteme sind heutzutage selten in Benutzung. Vertreter hiervon heißen JFFS, YAFFS, SPIFFS. Anstatt Daten direkt zu überschreiben, behandeln diese Systeme das Speichermedium als fortlaufendes, anhängendes Protokoll. Hier ist der Speicherort nicht an einen bestimmten Speicherblock gebunden. Mit einer Prüfsumme lässt sich ein Stromausfall leicht erkennen und auf den vorherigen Zustand zurücksetzen, indem man fehlgeschlagene Anhänge einfach ignoriert. Außerdem sorgt die zyklische Natur dafür, dass Logging-Dateisysteme den Verschleiß perfekt über den Speicher verteilen. Dies minimiert die Löschkzyklen auf Flash-Speichern und garantiert eine hohe Ausfallsicherheit. Nachteile sind die verhältnismäßig hohen Laufzeitkosten (häufig $O(n^2)$) und der hohe RAM-Verbrauch, was sie für Embeddedanwendungen eher ungeeignet macht.

4.3.3 Journaling-Dateisysteme

Bekannte und moderne Vertreter sind hier ext3, ext4 für Linuxbetriebssysteme oder NTFS für Windows. Diese Systeme schlagen die Brücke zwischen blockbasierten und logging-strukturierten Designs und haben große Vorteile, was die Korruption von Daten im Falle eines Stromausfalls angeht. Der Prozess findet im Falle von ext4 dabei konzeptionell vereinfacht wie folgt statt: Zunächst werden die für die Änderung wichtigen Informationen in das sogenannte Journal des Dateisystems geschrieben. Sobald diese Informationen vollständig auf dem Datenträger gespeichert wurden, wird die Transaktion durch einen Commit-Eintrag im Journal bestätigt. Anschließend werden die Änderungen an ihre endgültige Position im Dateisystem geschrieben. Nach erfolgreicher Übertragung wird der Commit-Eintrag nicht mehr benötigt und kann entfernt werden. Kommt es während dieses Vorgangs zu einem Systemabsturz, kann das Dateisystem beim nächsten Start das Journal auswerten. Transaktionen mit einem gültigen Commit-Eintrag können erneut an ihre endgültige Position geschrieben werden. Dadurch wird verhindert, dass eine Dateisystemänderung nur teilweise übernommen wird. Der Prozess ist wie erwähnt nur schematisch dargestellt, beispielsweise sichert ext4 standardmäßig nur seine Metadaten über Journals, trägt somit aber erheblich zur Konsistenz des Dateisystems bei[12].

Leider bringen Journaling-Dateisysteme einige Probleme mit sich. Sie sind ziemlich umfangreich in der Implementation, da im Grunde zwei Konzepte von Dateisystemen parallel benutzt werden, was zu Lasten der Codekomplexität geht. Zudem bieten sie aufgrund der engen Verbindung zwischen Speicherort und Daten keinen Schutz vor Verschleiß im Zusammenhang mit Flashspeichern. Dennoch sind sie seit Jahrzehnten der absolute Standard auf gewöhnlichen Desktop-Systemen.

4.3.4 Copy-on-Write (COW) Dateisysteme

Den letzten Typ von Dateisystemen bilden die copy-on-write (COW) Dateisysteme wie btrfs, ZFS oder das moderne Dateisystem von Windows ReFS. Diese sind den anderen blockbasierten Dateisystemen

sehr ähnlich, aber anstatt Blöcke direkt an Ort und Stelle zu aktualisieren, werden alle Aktualisierungen durchgeführt, indem eine Kopie der Änderungen erstellt und die Verweise auf den neuen Block umgelenkt werden.

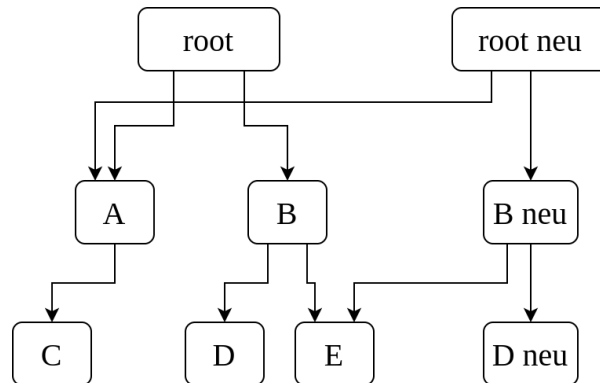


Abbildung 15: Funktionsprinzip COW nach der DESIGN.md

COW-Dateisysteme bieten eine langsamere, aber zumindest vergleichbare Performance zu blockbasierten Dateisystemen, während sie es gleichzeitig schaffen Aktualisierungen durchzuführen, ohne Datenänderungen direkt in einem Log oder Journal zu speichern.

Abbildung 15 zeigt das Prinzip der Organisation anschaulich. Daten werden bei COW-Dateisystemen oft in (balancierten) Bäumen gespeichert. Dies bezieht sich sowohl auf eine einzelne Datei, als auch auf z.B. Verzeichnisstrukturen. Der große Nachteil von COW-Dateisystemen ist, dass immer ein gesamter Zweig aktualisiert werden muss bis zur Wurzel, egal wie klein die Änderung ist. Wenn man beispielsweise Daten in Block D ändert, passiert folgendes: Es wird eine komplett neue Kopie erstellt namens „D neu“. Weil „D neu“ jetzt an einer ganz anderen Stelle auf dem Speichermedium liegt als das alte „D“, stimmt der Verweis im übergeordneten Ordner (Block B) nicht mehr. Also muss auch ein neuer Ordnerblock „B neu“ geschrieben werden, der auf „D neu“ zeigt. Weil jetzt aber „B neu“ an einer neuen Stelle liegt, stimmt der Verweis im Root-Block nicht mehr. Das System muss also zwingend auch einen brandneuen „root neu“-Block schreiben. Eine Aktualisierung kann also zu einer weitaus größeren Menge an Schreibvorgängen kaskadieren, als für die ursprünglichen Daten eigentlich nötig gewesen wäre. Dieses Hochwandern der Änderungen bis ganz nach oben zur Wurzel braucht $O(\log n)$ Schritte, wenn die Datei oder das System aus n Elementen besteht und in einem balancierten Baum organisiert ist.

4.4 FAT-Dateisysteme

Das FAT-Dateisystem wurde 1980 von Microsoft entwickelt und erstmals von MS-DOS unterstützt. Es wurde ursprünglich als einfaches Dateisystem entwickelt, das für Disketten mit einer Größe von weniger als 500 KB geeignet war. FAT ist die Abkürzung für File Allocation Table (Dateizuordnungstabelle). Derzeit gibt es vier FAT-Untertypen: FAT12, FAT16, FAT32 und seit 2006 exFAT als Nachfolger[8]. Im Laufe der Zeit wurden die Spezifikationen erweitert, um mit zunehmender Kapazität auch größere Speichermedien zu unterstützen.

Tabelle 3: Technische Adressierungsgrenzen der FAT-Dateisysteme[9]

Dateisystem	Max. Dateigröße	Max. Mediengröße	Hauptsächlicher zweck	Einsatz-
FAT12	16 MB	16 MB	Klassische 3,5-Zoll-Disketten (1,44 MB)	
FAT16	2 GB	2 GB	MS-DOS-Systeme, frühe MP3-Player, alte CNC- Maschinen	
FAT32	4 GB	2 TB	USB-Sticks, ältere Kameras oder Konsolen	
exFAT	512 TB oder mehr	512 TB oder mehr	Moderne SDXC/SDUC- Speicherkarten, externe SSDs	

Tabelle 3 zeigt die maximal möglichen, *üblichen* Datei- und Mediengrößen für die FAT-Dateisysteme und soll in erster Linie die Entwicklung der Spezifikation und die Probleme der älteren Dateisysteme darstellen. So ist es einem FAT32-Dateisystem beispielsweise nicht möglich eine 12 GB große .iso Datei aufzunehmen. Wie ergeben sich die maximalen Datei und Mediengrößen? Der Unterschied zwischen einem Sektor und einem Block wurde bereits erläutert: Der Sektor ist die physisch kleinste adressierbare Einheit, während der Block die logisch die kleinste adressierbare Einheit ist. Im Kontext der FAT-Dateisysteme wird der Block allerdings Cluster genannt, während der Begriff des Blocks meist im Unix-Kontext üblich ist. FAT12 stehen 12 Bits, FAT16 16 Bits und FAT32 28 Bits für die Adressierung der Cluster zur Verfügung. Mit größeren Clustergrößen ergibt sich also eine größere Mediengröße. Große Cluster haben allerdings den Nachteil, dass sehr kleine Dateien mindestens so groß werden wie die Clustergröße. Weiterhin wurden Clustergrößen immer gedeckelt, üblicherweise mit 32 KB Maximalgröße für FAT16 und FAT32. Dies führt bei FAT16 zu den genannten 2 GB und bei FAT32 zu 8 TB maximaler Mediengröße. Hinzu kommt allerdings eine weitere Limitierung: Der MBR enthält die Partitionstabelle. Dieser zählt allerdings in 512 Byte großen Sektoren und ihm steht lediglich ein Feld von 32 Bit für die Adressierung zur Verfügung. Folglich kann das FAT32-Dateisystem zwar eine größere Medien unterstützen, wird dann aber von der MBR-Partitionstabelle gebremst[9][8].

Die moderne Lösung dieser Limitierung liegt in der Verwendung des GUID Partition Table (GPT), die ebenfalls viele weitere Vorteile zum MBR bringt wie beispielsweise eine erhöhte Anzahl von Primärpartitionen und ein Backup der Partitionstabelle am Ende des Speichermediums [11].

Die Dateigröße bei FAT32-Systemen ist durch ein 32 Bit-Adressierungsfeld limitiert. Dieses zählt in Bytes, wodurch sich die charakteristischen 4 GB Dateigröße ergeben, die für heutige Zwecke manchmal nicht ausreichend sind[9][8]. Zu diesem Zweck wurde mit exFAT als moderne FAT-Variante 2006 eingeführt.

4.5 FatFs

4.5.1 Einleitung

FatFs ist ein plattformunabhängiges, quelloffenes Software-Modul zur Implementierung von FAT/exFAT-Dateisystemen in ressourcenbeschränkten, eingebetteten Systemen. Es wurde von dem Entwickler ChaN[7] entworfen und zeichnet sich durch einen extrem geringen Speicherbedarf sowohl im Programm- als auch im Arbeitsspeicher aus. Das Modul ist vollständig in ANSI C (C89) geschrieben. Dadurch ist es hardwareunabhängig auf praktisch jedem gängigen Mikrocontroller (z. B. AVR, PIC oder STM32) ohne Änderungen des Codes direkt lauffähig.

Die Architektur trennt das Dateisystem strikt in zwei logische Schichten: Das Application Interface stellt der Anwendung abstrakte, bekannte Funktionen zur Datei- und Verzeichnisverwaltung wie `f_open`, `f_read`, `f_write` und `f_close` zur Verfügung, während das Media Access Interface (Disk I/O) die Hardware vollkommen vom Dateisystem entkoppelt.

4.5.2 Implementierung

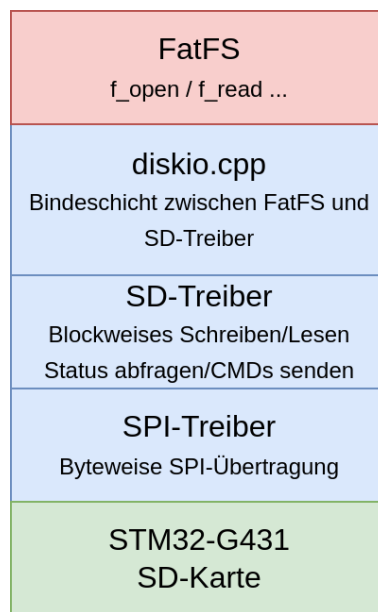


Abbildung 16: Implementierungsstack für das FatFS-Dateisystem

Abbildung 16 zeigt den vollständigen Implementierungsstack für die Verwendung von FatFs und damit für den Zugriff auf ein FAT-Dateisystem auf dem STM32G431. Alle in der Abbildung unterhalb von FatFs dargestellten Schichten müssen implementiert bzw. bereitgestellt werden. Die Datei `diskio.cpp` (in den meisten Implementierungen `diskio.c`) stellt dabei die von FatFs benötigte Schnittstelle zum darunterliegenden Blockgerät, hier SD-Karte, bereit. Sie übersetzt die von FatFs angeforderten Lese-, Schreib- und Statusoperationen in die entsprechenden Funktionen des verwendeten Treibers.

```

1  DSTATUS disk_initialize(BYTE pdrv);
2  DSTATUS disk_status(BYTE pdrv);
3  DRESULT disk_read(BYTE pdrv, BYTE* buff, LBA_t sector, UINT count);
4  DRESULT disk_write(BYTE pdrv, const BYTE* buff, LBA_t sector, UINT count);
5  DRESULT disk_ioctl(BYTE pdrv, BYTE cmd, void* buff);
6

```

Quellcode 12: Funktionen, die in der `diskio.cpp` implementiert werden mussten

Quellcode 12 zeigt die Funktionen, die für FatFs implementiert werden müssen. Zusätzlich musste auch noch `DWORD get_fattime()` implementiert werden, da jede FAT-Datei einen Zeitstempel braucht. Da die vorhandene Platine keine Echtzeituhr mitbringt, besitzt jede geschriebene Datei einen fixen Zeitstempel, der auf den 1. Januar 2026 datiert wurde. Die Funktion findet sich in der Datei `fatfs_time.cpp`, da sie thematisch nicht zu den anderen Funktionen in der `diskio.cpp` passt.

Bei den restlichen Funktionen ist die Hauptaufgabe die Funktionen aus der `sdcarddriver.hpp` korrekt aufzurufen.

```

1
2  template<typename Config>
3  DRESULT SD_disk_read(BYTE* buff, LBA_t sector, UINT count)
4  {
5
6      uint8_t result;
7
8      if (count == 1)
9      {
10         result = SD_ReadBlock<Config>(sector, buff);
11     }
12     else
13     {
14         result = SD_ReadBlocks<Config>(sector, count, buff);
15     }
16
17     //The SD_INIT_OK is my enum... the other RES stuff is FatFS, just to be clear
18     if (result != SD_READ_OK) return RES_ERROR;
19
20
21     return RES_OK;
22 }

```

Quellcode 13: Implementierung `SD_disk_read`

Quellcode 13 zeigt beispielhaft die Implementation von `SD_disk_read`, welches wiederum in der ursprünglichen Funktion `disk_read` aufgerufen wird. Da im Treiber Template-Funktionen verwendet werden, die ursprünglich von FatFs zur Verfügung gestellten Funktionen sich aber in einer `extern "C"` Umgebung befinden, ist ein zweistufiger Prozess mit einer Helferfunktion notwendig. In der eigentlichen `disk_read`

wird noch das Argument `BYTE pdrv` implementiert, was entscheidet, welches physische Laufwerk benutzt werden soll. In unserem Falle bleibt die Wahl zwischen den beiden SD-Kartenslots: Der SPI1-Peripherie bei `pdrv=0` und der SPI3-Peripherie mit `pdrv=1`.

Der *Quellcode* [13](#) hingegen zeigt, dass sich die benötigten Aufrufe für FatFs fast direkt in Funktionsaufrufe aus der `sdcarddriver.hpp` übersetzen lassen.

An dieser Stelle kommt es auch zu dem bereits erwähnten Namenskonflikt bei den Argumenten: So wird bei der Funktion `SD_ReadBlock` das Argument `sector` eingefügt, obwohl die Funktion das Argument `block` verlangt. In diesem Zusammenhang sind beide gleichbedeutend.

ffconf.h

Die Datei `ffconf.h` ist die zentrale Konfigurationsdatei des FatFs-Moduls. Über den C-Präprozessor lässt sich der Funktionsumfang auf die Hardware-Ressourcen des Mikrocontrollers zuschneiden.

- **Anpassbarer Funktionsumfang:**

- `FF_FS_READONLY`: Schaltet das Schreiben komplett ab, was Flash-Speicher einspart.
- `FF_FS_MINIMIZE`: Entfernt gezielt ungenutzte APIs wie z.B. die Verzeichnisoperationen (`f_mkdir`, `f_opendir`) oder Suchfunktionen.
- `FF_USE_MKFS`: Aktiviert oder deaktiviert die Formatierungsfunktion `f_mkfs`.
- `FF_USE_FASTSEEK`: Beschleunigt den wahlfreien Zugriff auf große Dateien.
- `FF_USE_FIND`: Aktiviert Suchfunktionen wie `f_findfirst()` und `f_findnext()`.

- **Dateinamen und Zeichenkodierung:**

- `FF_USE_LFN`: Schaltet die Unterstützung für lange Dateinamen frei.
- `FF_CODE_PAGE`: Legt die Codepage fest, um Sonderzeichen korrekt darzustellen.
- `FF_LFN_UNICODE`: Aktiviert Unicode-Unterstützung für Dateinamen.

- **Speicher- und Laufwerksoptionen:**

- `FF_VOLUMES`: Definiert die Anzahl der logischen Laufwerke, die gleichzeitig verwaltet werden können.
- `FF_MAX_SS` & `FF_MIN_SS`: Definiert die vom Dateisystem unterstützten logischen Sektorgrößen. Üblicherweise besitzen SD-Karten eine logische Sektorgröße von 512 Byte, FatFs kann jedoch auch größere Sektorgrößen unterstützen.
- `FF_FS_LOCK`: Legt fest, wie viele Dateien gleichzeitig vor konkurrierenden Zugriffen geschützt werden können.

- **Systemeinstellungen:**

- `FF_FS_TINY`: Reduziert den RAM-Bedarf pro Datei, indem interne Puffer gemeinsam genutzt werden.
- `FF_FS_EXFAT`: Aktiviert die Unterstützung des exFAT-Dateisystems für SDXC-Speicherkarten.

- `FF_FS_NORTC`: Verwendet feste Zeitstempel, wenn keine Echtzeituhr (RTC) vorhanden ist.
- `FF_FS_REENTRANT`: Aktiviert die Thread-Sicherheit des Dateisystems für Mehrthread-Anwendungen.

Darüber hinaus stellt FatFs zahlreiche weitere Konfigurationsoptionen bereit. Dazu gehören beispielsweise Einstellungen für mehrere Partitionen (`FF_MULTI_PARTITION`), Laufwerksbezeichnungen (`FF_STR_VOLUME_ID`), Dateiattribute (`FF_USE_CHMOD`), Laufwerkslabels (`FF_USE_LABEL`) oder die Unterstützung von TRIM-Befehlen für Flash-Speicher (`FF_USE_TRIM`). Dadurch lässt sich das Dateisystem flexibel an die Anforderungen der jeweiligen Embedded-Anwendung anpassen. Eine vollständige Auflistung findet sich einerseits direkt in der `ffconf.h` mit hilfreichen Kommentaren, andererseits auf der [offiziellen Webseite des Entwicklers](#). Aufgrund der verfügbaren Ressourcen des STM32G431 wurde auf eine weitgehend vollständige FatFs-Konfiguration zurückgegriffen, einschließlich der Unterstützung langer Dateinamen und des exFAT-Dateisystems.

4.6 LittleFS

4.6.1 Ziele von littlefs

Die Entwickler des littlefs-Dateisystems nennen auf ihrer Github-Seite drei große Vorteile ihres Dateisystems:

- **Ausfallsicherheit bei Stromausfall**
- **Verschleißregulierung des Flashspeichers (Wear Leveling)**
- **Begrenzter RAM/ROM-Bedarf**

Das Design von littlefs wurde in Hinblick auf die Gewährleistung dieser drei Hauptpunkte optimiert und soll im Folgenden näher beleuchtet werden.

4.6.2 Funktionsweise

Die Funktionsweise von littlefs wird in der bereits zitierten [DESIGN.md](#) erläutert. Diese ist äußerst komplex und kann in diesem Rahmen nur schemenhaft erläutert werden: Die „Superblock-Ebene“, also die oberste Strukturebene des Dateisystems, nutzt eine modifizierte Form des bereits erläuterten COW (Copy-on-Write) für die grundsätzliche Neubeschaffung und Strukturierung von Blöcken. Das sogenannte CObW (Copy-On-bounded-Write) führt einen Zähler ein, sodass nicht bei jeder Modifikation der gesamte Baum neu geschrieben werden muss wie bei COW, sondern nur alle N male. Bis zur N-ten Modifikation bleibt der alte Baum gültig. Dies reduziert den Aufwärtsskaskadierungs Aufwand, den COW-Dateisysteme mit sich bringen, um den Faktor N. CObW wird bei littlefs also für die größten Datenstrukturen verwendet und bringt den Vorteil des Wear-Leveling mit sich, sowie eine Dateikonsistenzgarantie bei unerwartetem Stromausfall.

Auf Dateiebene nutzt littlefs unterschiedliche Mechanismen für die tatsächlichen Dateiinhalte und die Metadaten (Dateiname, Größe, etc.). Für den Dateiinhalt wird eine sogenannte „CTZ skip-list“ verwendet, die man sich als eine Erweiterung der Datenstruktur der verketteten Liste vorstellen kann.

Der große Unterschied ist, dass die „skip-list“ zusätzliche Zeiger einführt, die Elemente überspringen können. In einer gewöhnlichen „skip-list“ ist es zufallsbasiert, wie viele Zeiger ein bestimmter Eintrag

Tabelle 4: Zahl von Zeigern in der „CTZ skip-list“ in Abhängigkeit des Blockindexes.

Blockzahl	Blockzahl(bin)	CTZ	Zeiger
1	0001	0	1
2	0010	1	2
3	0011	0	1
4	0100	2	3
5	0101	0	1
6	0110	1	2
7	0111	0	1
8	1000	3	4

hat. Die CTZ-skip-list (Count Trailing Zeros) nutzt als Zeigerverteilung einfach die Anzahl der Nullen einer Zahl in Binärdarstellung. *Tabelle 4* zeigt die Verteilung beispielhaft für die Blöcke 1 bis 8. Der Vorteil dieser Datenstruktur ist die Suchlaufzeit von $O(\log n)$ für einen speziellen Block, während die Traversal durch alle Blöcke einer Datei in einer Laufzeit von $O(n)$ möglich ist und ein Anhängen eines Blocks an eine Datei in $O(1)$. Eine rückwärts verkettete Liste hätte beispielsweise den Nachteil, dass die Traversal durch alle Blöcke $O(n^2)$ Schritte benötigt, während eine vorwärts verkettete Liste nicht an eine Datei Blöcke anfügen kann ohne mindestens einen vorherigen Block zu modifizieren. Andere beliebte Datenstrukturen wie balancierte Bäume haben Nachteile bezüglich einer beschränkten RAM-Nutzung, wie sie bei eingebetteten Systemen gefordert ist.

Für Metadaten wird ein „two-block log“ verwendet.

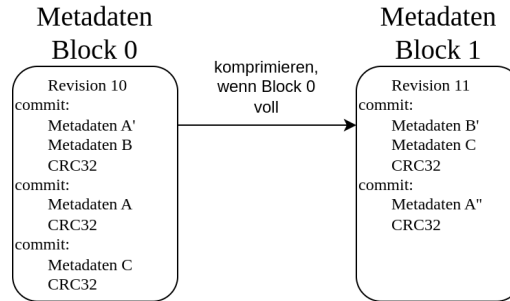


Abbildung 17: Prinzip „two-block log“

Abbildung 17 zeigt die Idee hinter dem „two-block log“ für Metadaten auf. Die Metadaten stehen zu jedem Zeitpunkt in zwei Blöcken, die in sich eine begrenzte Anzahl an commits zulassen. Ein commit kann beispielsweise eine Änderung der Metadaten beinhalten oder ein Hinzufügen von Metadaten. Jeder commit wird mit einer CRC32 abgeschlossen; Ist diese nicht gültig, so wird der commit verworfen. Das Prinzip ist also das eines Logging-Dateisystems, allerdings beschränkt in der Größe. Sobald einer der Blöcke voll ist, wird er auf den anderen Block komprimiert. Diesem wird dann eine neue Revisionsnummer gegeben und fortan ist dies der neue aktive Logblock. Die Vorteile, die das System mit sich bringt ist die dauerhafte Konsistenz der Metadaten, da nichts überschrieben wird, sondern zu jedem Zeitpunkt eine korrekte Version der Daten existiert. Weiterhin bringt das Logging durch die Begrenzung der Größe des Logs eine nahezu konstante Laufzeit mit sich. Dadurch, dass aber überwiegend mit Logs gearbeitet wird, ist fundamental der Anspruch an Wear-Leveling erfüllt. Zum Vergleich: FatFs überschreibt seine Metadaten an Ort und Stelle und gefährdet damit die gesamte Dateintegrität, falls die Schreiboperation scheitert. Ebenfalls ist

das nicht optimal für das Wear-Leveling.

Grundsätzlich gilt für Metadaten bei littlefs der Begriff der Atomarität: Entweder die Operation wird vollständig ausgeführt oder gar nicht. Die CRC32-Prüfsumme gewährleistet die Vollständigkeit eines commits.

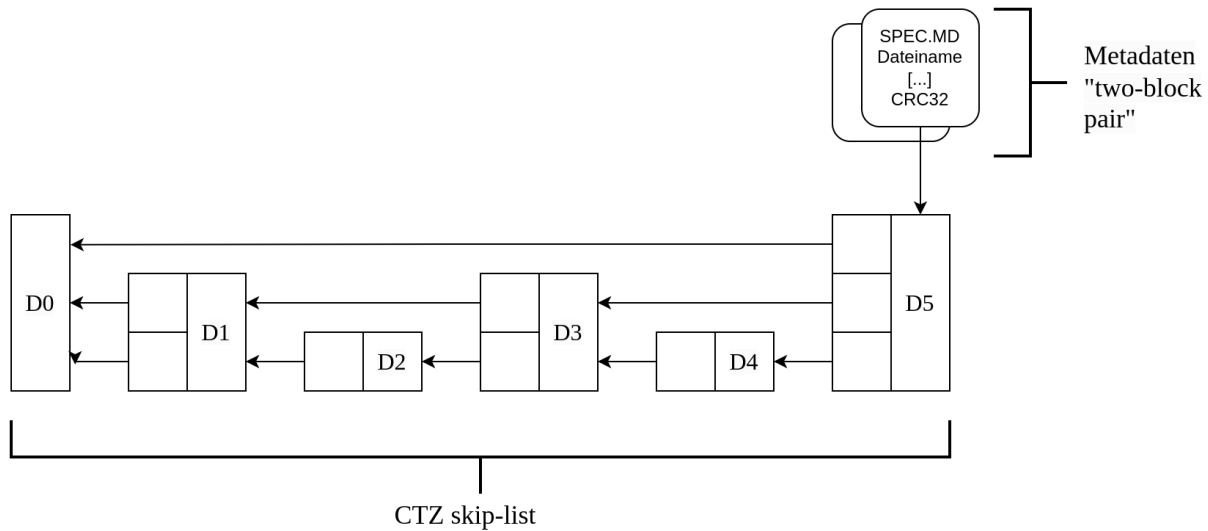


Abbildung 18: Prinzip des Dateiaufbaus bei littlefs

Abbildung 18 zeigt anschaulich den prinzipiellen Aufbau einer Datei in littlefs nach dem erläuterten Prinzip.

Ein wichtige, wenn auch in der DESIGN.md nur am Rande erwähnte Funktionalität, ist das Zurücklesen des geschriebenen Buffers und ein Vergleichen der Nachricht. Damit kann littlefs feststellen, ob das Schreiben tatsächlich geklappt hat oder doch möglicherweise ein Schreibfehler vorliegt. Im Falle vom Flashspeicher könnte in diesem Falle ein Badblock vorliegen, also ein defekter Speicherblock. Das Dateisystem probiert daher das Schreiben auf einem anderen Block und zwar solange, bis es die Nachricht korrekt zurücklesen kann.

4.6.3 Implementierung

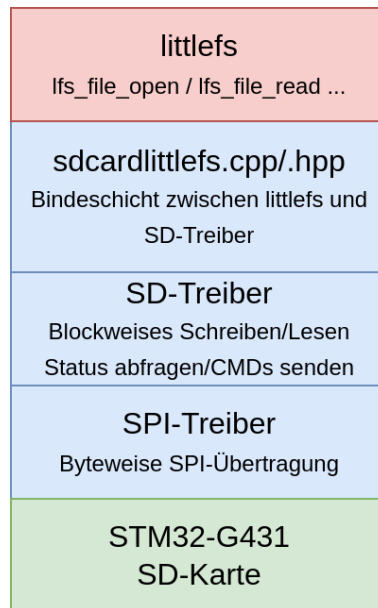


Abbildung 19: Implementierungsstack für das littlefs-Dateisystem

Abbildung 19 zeigt den Implementierungsstack für das littlefs-Dateisystem. Im Grunde genommen nutzt littlefs genauso wie FatFs die Blockschreib- und Lesefunktionen des SD-Kartentreibers. Die Datei, die littlefs und SD-Kartentreiber verbindet, findet sich unter `src/sdcardlittlefs.cpp`. Leider besteht die hauptsächliche Dokumentation für littlefs aus Kommentaren in der Quelldatei `littlefs/lfs.h`. Zusätzlich gibt es auf der GitHub-Seite ein unvollständiges Beispiel und die bereits erwähnte ausführliche Erklärung des Prinzips in der [DESIGN.md](#). Kurzum sind die vorhandenen Informationen im Gegensatz zu FatFs sehr beschränkt, sodass zwar eine funktionierende, aber möglicherweise keine optimale Implementierung der Bindeschicht in der `src/sdcardlittlefs.cpp` gewährleistet werden konnte.

4.6.4 littlefs FUSE wrapper

Während sich mit FAT formatierte SD-Karten problemlos an jedem Desktop-PC mit jedem erdenklichen Betriebssystem auslesen lassen, existiert für littlefs das [littlefs FUSE wrapper](#) Projekt. Nach erfolgreichem Kompilieren des Projektes lässt sich unter Linux beispielsweise mit `sudo ./lfs /dev/mmcblk0 mount/ -o allow_other` das Dateisystem einhängen und mit kleinen Einschränkungen nutzen. Zu den Einschränkungen gehört, dass littlefs keine Zeitstempel für Dateien anlegt und auch kein System für spezielle Berechtigungen existiert.

An dieser Stelle stellt sich für die Praxis ebenfalls die Frage, ob der Endnutzer mit dem Dateisystem in Berührung kommen wird. Während die Einrichtung des littlefs FUSE wrapper Projektes für technisch versierte und mit GitHub vertraute Personen problemlos in unter einer Stunde möglich ist, ist es für normale Endnutzer ungeeignet und überfordernd. Littlefs ist also kein Dateisystem, was für Geräte wie beispielsweise Digitalkameras oder Oszilloskope zu empfehlen wäre, wo dem Endnutzer eine garantierte Kompatibilität mit dem Dateisystem garantiert werden sollte.

5 Vergleich littlefs, FatFs und direkter Blockadressierung

Im Folgenden wird die direkte Blockadressierung als „NoFs“ bezeichnet.

5.1 RAM- und Flashverbrauch

Tabelle 5 ist die Auswertung der Ausgabe des Compilers „arm-none-eabi-gcc“ bei unterschiedlichen Optimierungsstufen. Für FatFs wurde eine kleine Standardkonfiguration verwendet. Das bedeutet konkret, dass alle Standardfunktionen vorhanden sind (FF_FS_MINIMIZE=0), aber der Support für lange Dateinamen (FF_USE_LFN=0) abgeschaltet ist. Der statische RAM-Verbrauch hält sich über alle Dateisysteme recht stabil im Bereich von 2 KB und schwankt auch nach der Compileroptimierung nicht sonderlich. Hier von ausgenommen ist FatFs, dessen statischer RAM-Verbrauch in optimierten Builds um fast 500 kB sinkt im Vergleich zum unoptimierten Build, sodass lediglich 1,5 KB an statischem RAM verbraucht werden. Da auch FatFs intern dieselben Treiber verwendet wie NoFs und littlefs, ist dies ein Hinweis darauf, dass die internen Buffer für FatFs auf dem Stack allokiert werden, während NoFs und littlefs statische Buffer nutzt, die der Compiler beim Build im .data oder .bss Segment verordnen und somit anzeigen kann.

Weitere Untersuchungen des FatFs-Dateisystems haben gezeigt, dass sich der Hauptbuffer von FatFs in der Struktur FIL befindet und diese eine Größe von 552 Bytes aufweist. Ob der Buffer also auf dem Stack liegt oder im statischen/globalen Segment allokiert wird liegt in diesem Fall also in der Hand des Programmierers, was für Systeme, die ihren RAM-Bereich fest aufteilen, sehr nützlich sein kann. Dies ist selbstverständlich auch der Fall für NoFs.

Der Flashverbrauch entspricht den Erwartungen: Er ist bei NoFs am kleinsten und bei littlefs am höchsten. Das hängt mit der Komplexität des Dateisystems zusammen. Während FatFs ein relativ simples, blockbasiertes Dateisystem ist, fusioniert littlefs die Ideen der Journaling and Copy-on-Write Dateisysteme für mehr Ausfallresilienz und Wear-Leveling. Dies spiegelt sich im Speicherverbrauch für den Programmspeicher wider. NoFs verwendet in der kleinsten Variante etwa 2 KB, FatFs etwa 15 KB und littlefs etwa 35 KB Speicher auf dem Flash. Während das für den verwendeten STM32-G431 mit 128 KB Flashspeicher kein Problem darstellt, so gibt es zahlreiche kostengünstige STM-Modelle wie beispielsweise Modelle aus der STM32C0-Serie, die nur über 32 kB Programmspeicher verfügen und somit kein littlefs in dieser Konfiguration nutzen können.

Ebenfalls interessant ist zu sehen, dass die Programmgröße bei der Optimierung „O3“ größer ist als bei „O2“. Dies hängt damit zusammen, dass auf Ausführungsgeschwindigkeit optimiert wird statt auf Größe. So werden beispielsweise Funktionen aggressiver geinlined, um Funktionsaufrufe zu sparen, was sich aber wiederum in einer höheren Programmgröße widerspiegelt.

Tabelle 5: Flash und statischer RAM der verschiedenen Dateisysteme bei unterschiedlicher Compiler-Optimierungsstufe

Optimierungsstufe	Dateisystem	Flash (Bytes)	Statischer RAM (Bytes)
-O0	littlefs	57504	2184
	NoFs	13424	2512
	FatFs	28304	2008
-O1	littlefs	39716	2168
	NoFs	3120	2080
	FatFs	19316	1576
-O2	littlefs	39308	2168
	NoFs	2404	2080
	FatFs	17536	1576
-O3	littlefs	47596	2168
	NoFs	2720	2080
	FatFs	18612	1576
-Os	littlefs	35148	2168
	NoFs	2124	2080
	FatFs	15044	1576

5.2 Versuch 1: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge

```
1  for(;;)
2  {
3      int tcnt_ms = TIM2->CNT/1000;
4      std::snprintf(filename, sizeof(filename), "l_%06d_%06d_%08d.txt",
5      file_number, cnt_number, tcnt_ms);
6
7      char msg[32];
8      int msg_len = std::snprintf(msg, sizeof(msg), "Write %08d!\n", file_number);
9      for (int j = 0; j < 64; j++)
10     {
11         std::memcpy(lfswritebuf + j * msg_len, msg, msg_len);
12     }
13
14     int res_open = lfs_file_open(&lfs_inst, &file, filename, LFS_O_RDWR | LFS_O_CREAT);
15     if(res_open == LFS_ERR_OK)
16     {
17         lfs_file_write(&lfs_inst, &file, &lfswritebuf, sizeof(lfswritebuf));
18
19         lfs_file_close(&lfs_inst, &file);
20         file_number++;
21     }
22     cnt_number++;
23 }
```

Quellcode 14: Hauptschleife aus test/littlefs_write_test.cpp

Quellcode 14 zeigt die relevante Hauptschleife des Schreibtests für das littlefs-Dateisystem. Der Schreibtest für das FatFs-Dateisystem ist analog dazu in test/fatfs_write_test.cpp vorhanden. Abbildung 20 verdeutlicht den Ablauf anschaulich. Die aufgenommenen Testdaten finden sich im GitHub-Projektverzeichnis unter (root)/test_data/Writetest1/. Die Auswertung folgte am stationären Desktop-Computer. Dazu wurden kurze Pythonskripte erstellt, die Dateigröße und Integrität der Daten prüfen. Das relevanteste Skript ist hierbei (root)/test_data/Writetest1/test_readable.py. In der Auswertung zeigt sich, dass unter etwa 9000 littlefs-Dateien und rund 8000 FatFs-Dateien keine einzige Datei auch nur einen einzigen messbaren Bitfehler aufweist, trotz fehlender CRC-Prüfsumme bei der SPI-Datenübertragung. Somit kann anhand dieses Versuchs keine Aussage darüber getroffen werden, ob ein Dateisystem zuverlässiger mit Daten umgeht als das andere. Gleichzeitig bestätigt der Versuch, dass die zugrundeliegenden Blockschreibfunktionen über eine Vielzahl Aufrufe zuverlässig arbeiten.

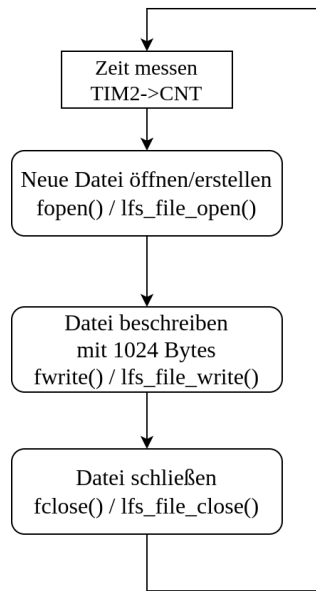


Abbildung 20: Schreibzyklus für eine 1024 Byte große Versuchsdatei

Tatsächlich war der Plan etwa die zehnfache Menge an geschriebenen Daten für diesen Versuch zu erzeugen. Dies war durch kurze Messreihen von einer Minute abgeschätzt worden. Während des Schreibversuches ist klar geworden, dass deutlich weniger Dateien erstellt wurden als zunächst angenommen. Während in dieser Arbeit zwar vorrangig die Zuverlässigkeit der Dateisysteme bezüglich der Datenintegrität untersucht werden sollte, ist es beispielsweise für Logginganwendungen unverzichtbar ein konsistentes Verhalten beim Erstellen und Schreiben von Dateien zu erreichen. Also wurde der Versuch um einen Zeitstempel im Dateinamen erweitert, sodass die Dauer des gesamten Zyklus der Dateierstellung wie in *Abbildung 20* aufgezeigt, gemessen werden konnte.

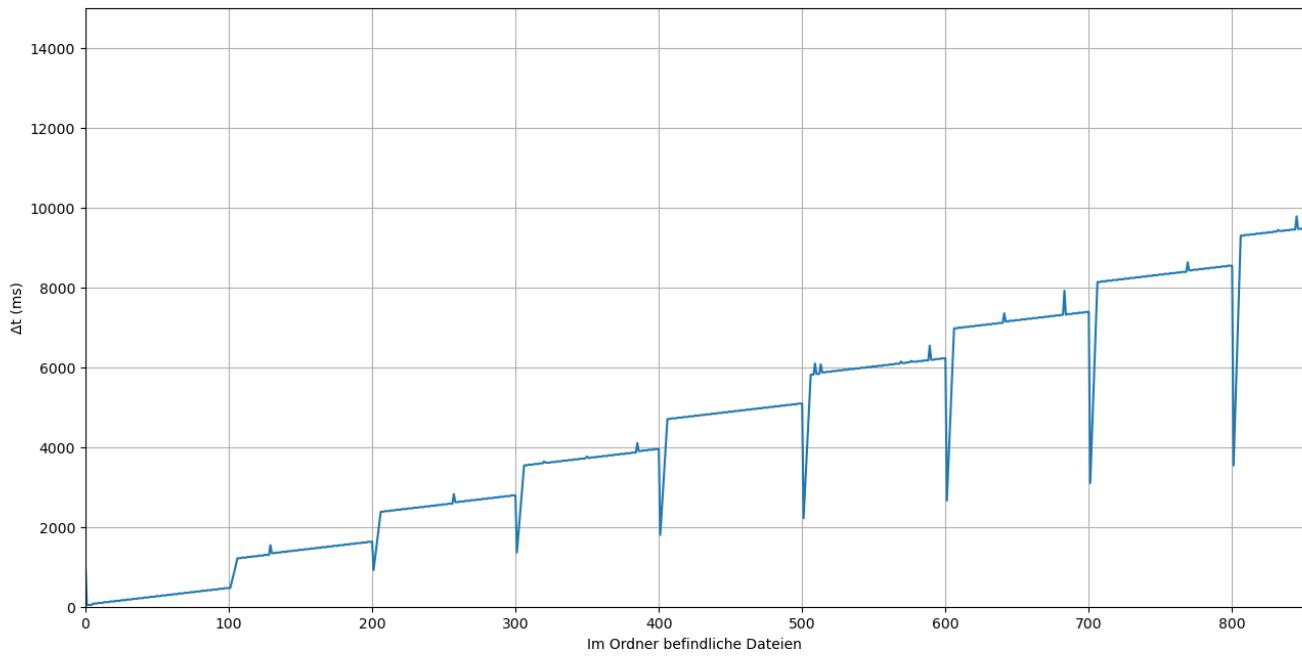


Abbildung 21: Dauer der Neuerstellung von Dateien auf dem FatFs-Dateisystem

Abbildung 21 zeigt die Dauer des Schreibvorgangs für eine 1024 Byte Datei in Abhängigkeit davon wie viele Dateien bereits erstellt wurden. Es zeigt sich ein klare lineare Abhängigkeit mit einem auffällig periodisch wiederkehrendem Muster.

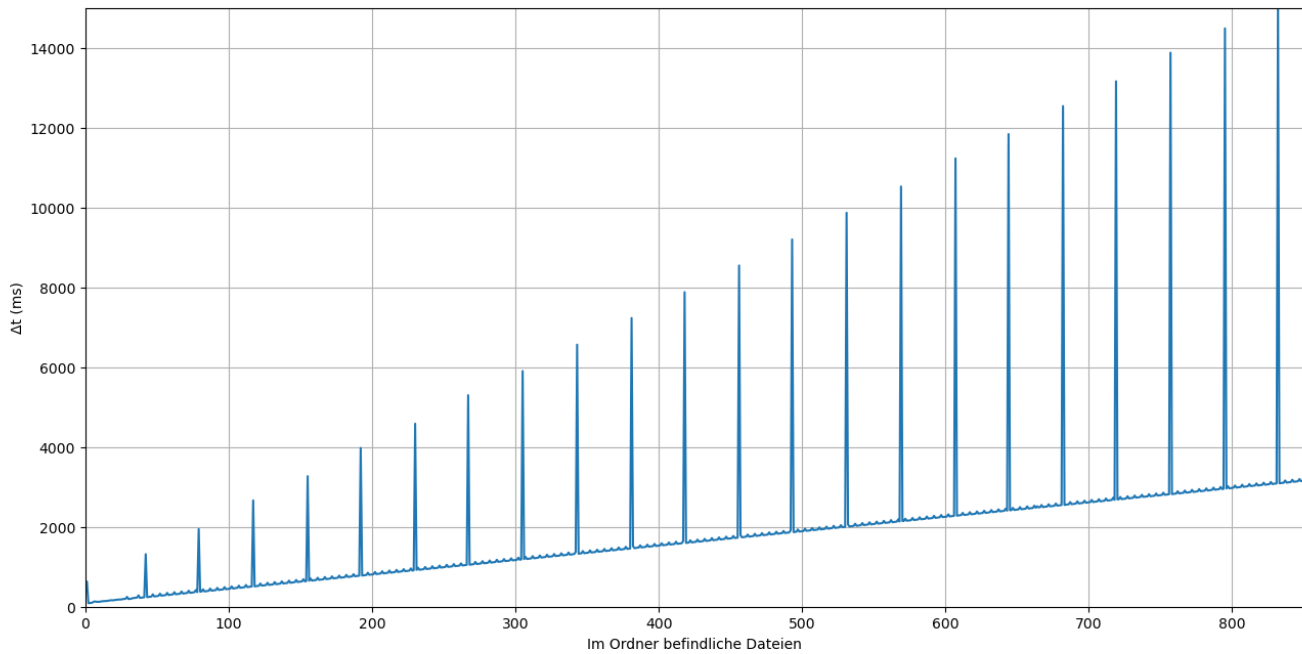


Abbildung 22: Dauer der Neuerstellung von Dateien auf dem littlefs-Dateisystem

Abbildung 22 zeigt dieselbe Messreihe für das littlefs-Dateisystem. Auch hier sehen wir einen konstanten, linearen Anstieg der Dateierstelldauer in Abhängigkeit der bereits vorhandenen Dateien. Zusätzlich braucht das Dateisystem alle 38 Dateien einmalig deutlich länger für die Erstellung einer Datei. Eine mögliche Erklärung für dieses Verhalten könnte der bereits erwähnte COBW-Mechanismus sein. Nach 38 Dateien wäre der Zeitpunkt erreicht, wo einmalig ein ganzer Zweig eines Baumes neu geschrieben werden muss. Während hier der Übersichtlichkeit halber nur ein Auszug von rund 850 Dateien dargestellt ist, wurden alle vorhandenen Messdaten untersucht. Sowohl für littlefs, als auch für FatFs setzt sich das typische Muster mit linear steigender Dateierstelldauer fort.

Bei genauer Betrachtung der Werte fällt auf, dass das FatFs-Dateisystem trotz geringerer Dateisystemkomplexität länger für die Erstellung von Dateien benötigt als littlefs. Die Vermutung lag nahe, dass das mit FatFs Option „LFN“, „Long File Names“ zusammenhängt. Dies ist eine Erweiterung für lange Dateinamen mit Unicode-Kodierung, die im klassischen FAT-Standard zunächst nicht vorgesehen war. Diese Form der Dateinamen belegt insgesamt mehr Speicher aufgrund komplexerer Kodierung. Es wurden daher noch einmal eine Messreihe mit FatFs im klassischen „SFN“, „Short File Name“ oder auch 8.3 Format, aufgenommen. Das Problem an diesem Format ist, dass nicht nur keine Umlaute für Dateinamen funktionieren, sondern auch noch alle Dateien in Großbuchstaben aufgezeichnet werden und nur eine maximale Länge von 8 Zeichen aufweisen dürfen, sowie 3 weitere Zeichen für Dateiendungen wie „.txt“. Abbildung 24 bestätigt die Vermutung. Mit dem klassischen SFN-Format zeigt FatFs eine deutlich bessere Performance beim Erstellen von Dateien als littlefs.

Fazit Versuch 1

Auch wenn in diesem Versuch keine Aussage über die Datenintegrität getroffen werden konnte, da die zugrundeliegende Übertragungsebene fehlerfrei gearbeitet hat, so lassen sich dennoch wichtige Schlüsse für den Umgang mit Dateien auf eingebetteten Systemen ziehen: Bei der Speicherung von Daten ist es für die Geschwindigkeit des Schreibzyklus vorteilhaft diese in möglichst wenig Dateien zu bündeln. Sollen dennoch separate Dateien erzeugt werden, so ist es vorteilhaft diese ab einer gewissen Anzahl auf Unterordner aufzuteilen. Eine mögliche Erklärung ist hierfür, dass die Dateisysteme vor dem Erstellen neuer Dateien den gesamten Inhalt des Ordners zunächst auslesen müssen. Wenn dieser eine komplexe Kodierung enthält wie im Falle von Fat32-LFN oder sehr viele Dateien, so schadet das der Performance, insbesondere da auf eingebetteten Systemen die Speicherbuffer sehr klein sind mit denen gearbeitet wird.

5.3 Versuch 2: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge und manipulierter Schreibfunktion

Versuch 2 folgt im Aufbau und in der Idee analog zu Versuch 1. Aufgrund der hohen Zuverlässigkeit der Schreibfunktionen wird nun allerdings eine gezielte Fehlerinjektion in den Schreibfunktionen des SD-Kartentreibers durchgeführt, bei der nach jeweils 5000 Bytes ein Byte modifiziert wird. Da sowohl littlefs, als auch FatFs diese Funktion als Basis nutzen, ist eine Überprüfung des Umgangs mit fehlerhaften Daten auf Dateisystemebene möglich.

```
1 // send 512 byte blockdata
2 for (uint16_t i = 0; i < 512; i++) {
3     //Injecting errors
4     if(err_cnt == 5000)
5     {
6         //some random data
7         //const_cast<uint8_t*>(buffer)[i] = 0xAA;
8         SpiDriver<Config::SpiBase>::Transfer(0xAA);
9         err_cnt = 0;
10        continue;
11    }
12    err_cnt++;
13    SpiDriver<Config::SpiBase>::Transfer(buffer[i]);
14 }
```

Quellcode 15: Modifizierter Teil der Funktion SD_WriteBlock() aus Datei `src/sdcarddriver.hpp`

Quellcode 15 zeigt den modifizierten Teil der Schreibfunktion. Die aufgenommenen Rohdaten finden sich in [\(root\)/test_data/Writetest2/](#).

In der Auswertung zeigt sich, dass littlefs das Datei- und Datenintegritätsversprechen hält: Rund 500 Dateien konnten mit fehlerfreiem Dateinamen und Inhalt geschrieben und überprüft werden. Während die Metadaten durch das komplexe CRC32 Logging-System abgesichert sind, konnten alle Fehler beim Dateiinhalt durch das integrierte Zurücklesen des geschriebenen Blockes aufgedeckt werden. FatFs hingegen

liest die geschriebenen Daten nicht noch einmal ein und überprüft sie auch nicht auf Korrektheit.

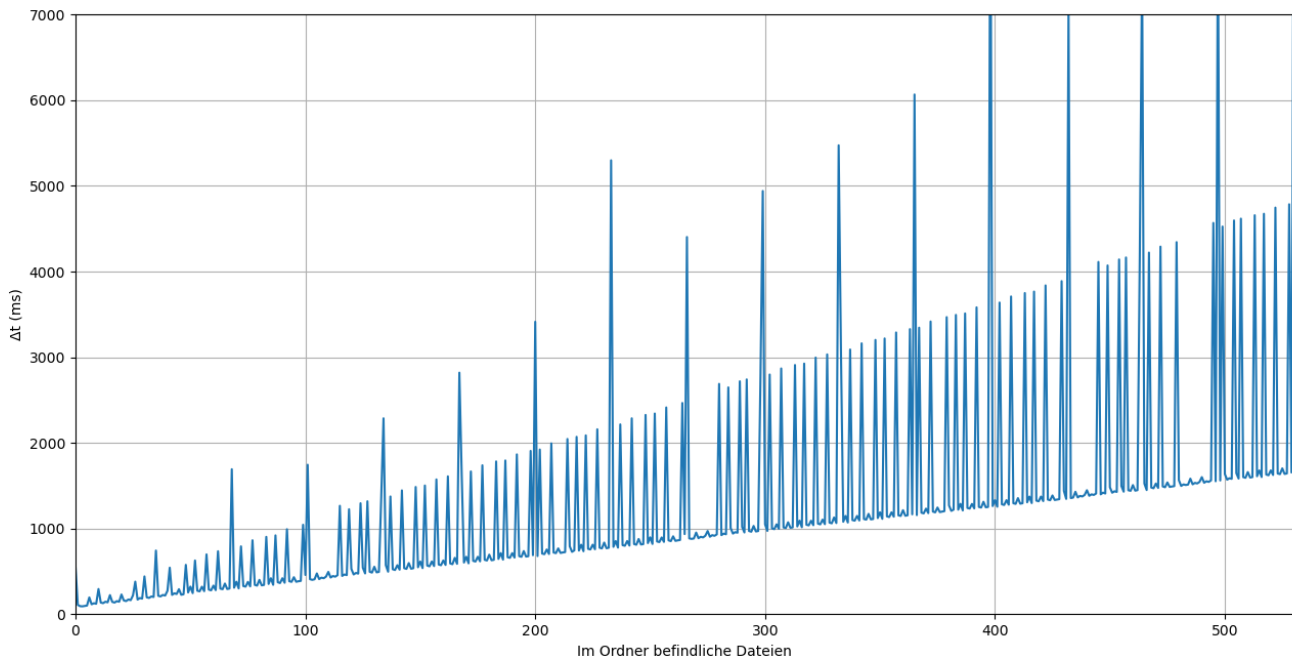


Abbildung 23: Dauer der Neuerstellung von Dateien auf dem littlefs-Dateisystem mit korrupten Daten

Abbildung 23 zeigt den aus Versuch 1 bekannten Graphen, in dem die Dauer gezeichnet wird, die die Neuerstellung einer Datei in Abhängigkeit von den sich bereits im Ordner befindlichen Dateien benötigt. Logischerweise benötigt littlefs viel häufiger längere Zeit zur Erstellung einer Datei als zuvor, da es nach der Fehlerprüfung die Daten neu anfordern und neu senden muss.

Bei FatFs hingegen fällt allein schon die automatische Auswertung schwer: Durch Fehler sowohl beim Dateninhalt, als auch in den Metadaten (Dateiname, Zeitstempel, Dateigröße etc.) lässt sich das zuvor verwendete Skript nicht zuverlässig anwenden. Von 337 Dateien haben mindestens 28 Formatierungsfehler im Dateinamen, 8 Dateinamen sind dabei völlig unkenntlich. 66 Dateien haben jeweils exakt einen Schreibfehler im Inhalt. Zusätzlich gab es Fehler bei der Dateigröße und im Zeitstempel. Diese wurden leider beim Kopieren in den Testordner bereits wieder beseitigt, da sie beim Kopieren auf ein neues Dateisystem überführt wurden.

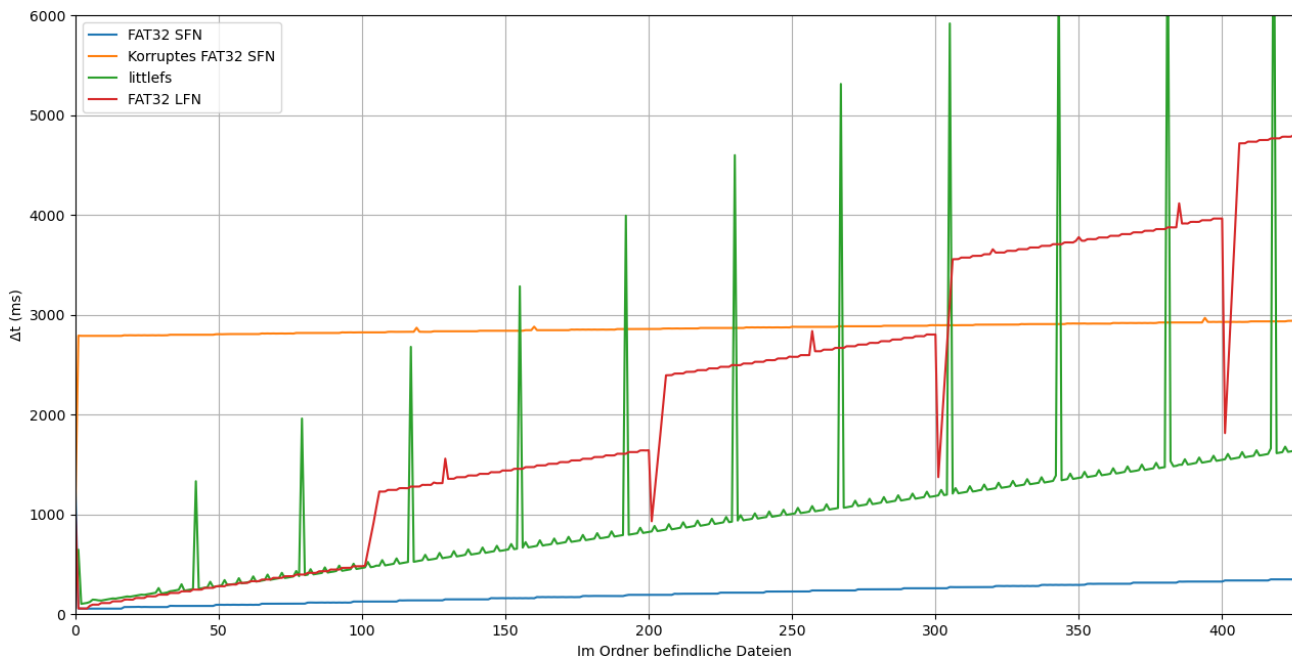


Abbildung 24: Dauer der Neuerstellung von Dateien Übersicht

Abbildung 24 zeigt eine Übersicht über die Dateierstellzeiten in verschiedenen Kontexten. Interessant ist hier der Graph für „Korruptes FAT32 SFN“. Ein Löschen der korrupten FAT32 Dateien war nämlich nur bedingt möglich. Das zurückgebliebene inkonsistente Dateisystem hat verglichen mit einem frisch formatierten FAT32 erhebliche Performanceprobleme. An dieser Stelle sei erwähnt, dass alle Graphen in Abbildung 24 im Gegensatz zu Abbildung 23 mit einer unmanipulierten Schreibfunktion erstellt wurden. Die Zeitverzögerung für den Graphen „Korruptes FAT32 SFN“ entsteht lediglich durch ein inkonsistentes Dateisystem. Dieses kann nur durch Neuformatierung oder durch spezialisierte Software wie „fsck (file system check)“ wieder repariert werden.

Fazit Versuch 2

Bei einem unsicheren Übertragungskanal für die Daten bedarf es für das FAT-Dateisystem einer gesonderten Strategie, um die Integrität der Daten und des Dateisystems zu sichern. Ganz trivial ist das allerdings nicht: Wenn die Metadaten korrupt sind, so ist beispielsweise ein Öffnen der Datei mit der `fopen()`-Funktion von FatFs gar nicht möglich, da der Dateiname nicht stimmt. Viel einfacher scheint hier die Einführung einer neuen, sicheren Schreibfunktion auf SD-Treiberebene, die das Zurücklesen direkt übernimmt und im Zweifelsfall die Operation wiederholt.

Littlefs hingegen bringt durch die im Kapitel Funktionsweise littlefs besprochenen Schutzmechanismen hier klare Vorteile mit sich.

5.4 Versuch 3: Analyse von pseudo-randomisierter Zeichenfolge bei Anhang an Datei

Die Idee dieses Versuches ist einerseits die Konsistenz der Schreiboperationen bei pseudo-zufälligen Daten festzustellen, andererseits die Schreibperformance bei dem Anhängen von Daten an eine bereits existierende Datei im Gegensatz zur Neuerstellung der Datei zu untersuchen. Weiterhin wurde der SPI-Takt auf 8 MHz bei diesem Versuch erhöht, um mehr potentielle Schreibfehler zu provozieren.

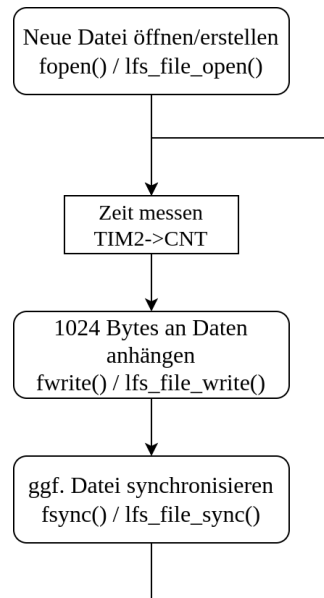


Abbildung 25

Bisher wurde primär die Neuerstellung von kleinen Dateien der Größe 1 KB untersucht. Es wurde in den vorherigen Versuchen festgestellt, dass der Zeitaufwand der Neuerstellung einer Datei linear zunimmt bei beiden Dateisystemen. Für die dauerhafte Aufnahme von Daten ist das sehr nachteilhaft: Für eine Echtzeitgarantie bei der Aufnahme von Daten müsste beachtet werden wie viele Dateien sich bereits im Ordner befinden und gegebenenfalls müsste ein neuer Ordner erstellt werden. Daher wurde mit den Tests `test/fatfs_append_write_test.cpp` und `test/littlefs_append_write_test.cpp` die Zeitdauer aufgenommen, die das Anhängen von 1 KB Daten an eine bestehende Datei benötigt.

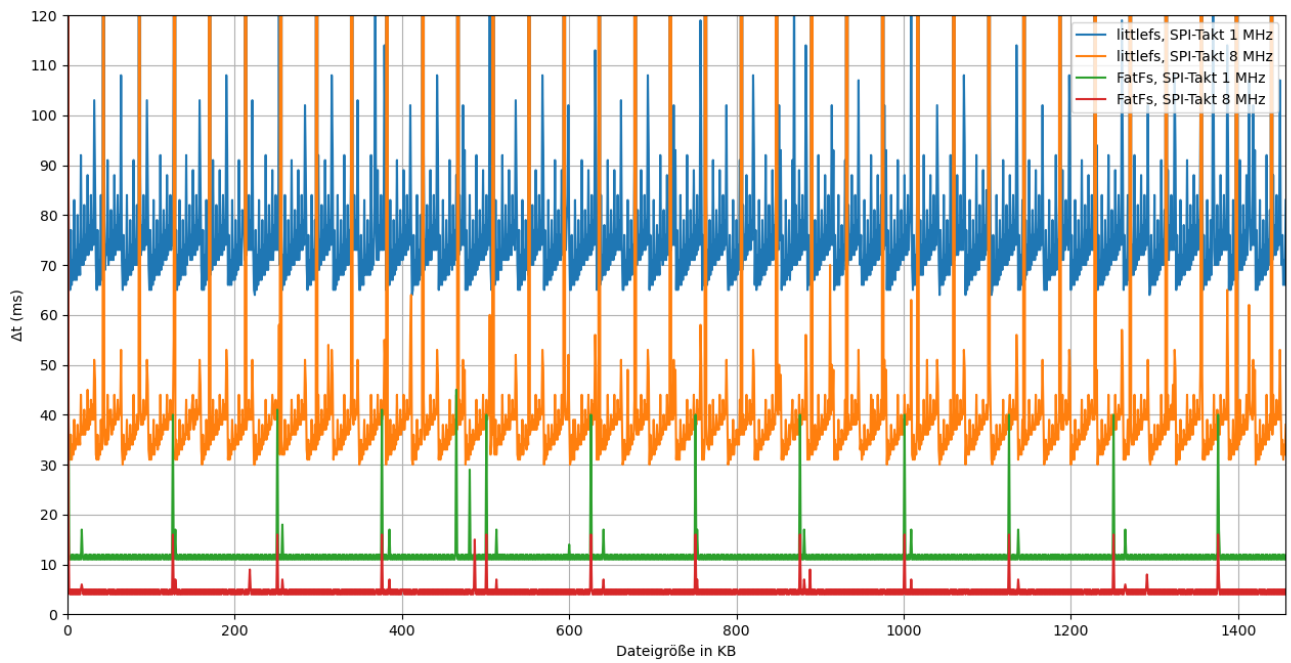


Abbildung 26: Dauer des Anhängens von 1 KB an eine Datei in Abhängigkeit der Dateigröße

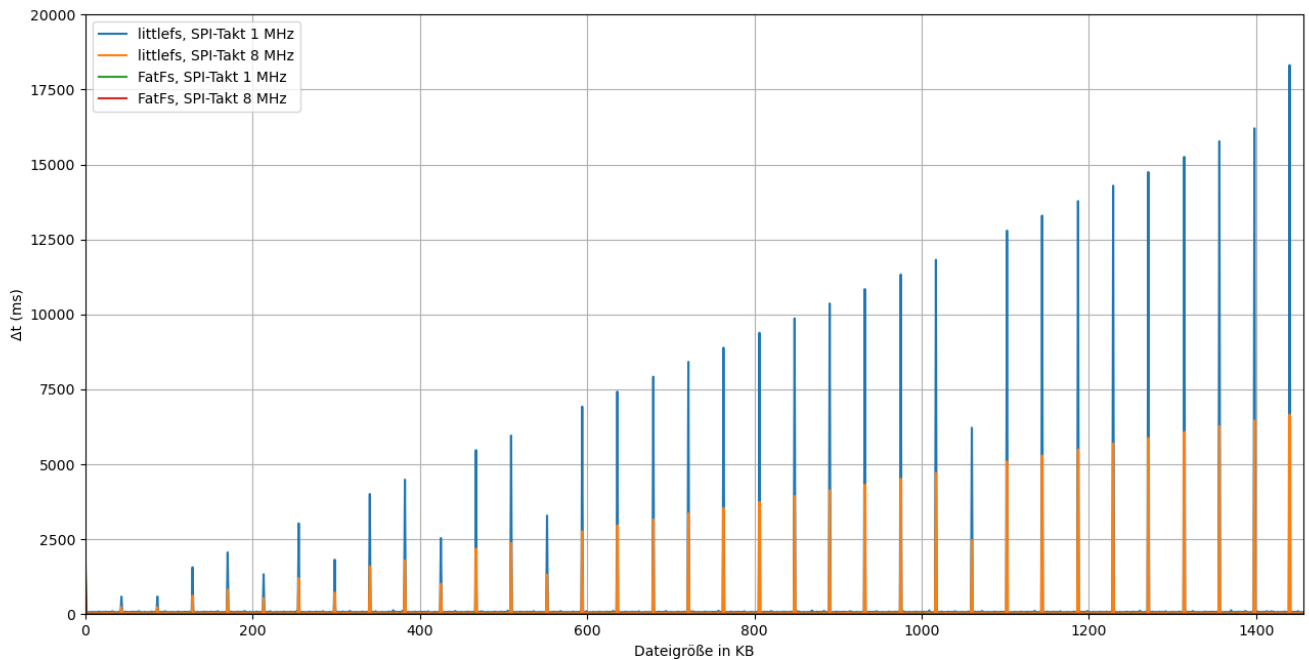


Abbildung 27: Dauer des Anhängens von 1 KB an eine Datei in Abhängigkeit der Dateigröße

Abbildungen 26 und 27 zeigen anschaulich das Verhalten der beiden Dateisysteme und nutzen denselben

Datensatz in unterschiedlicher Skalierung. Während das Anhängen von 1 KB Datengröße an eine Datei bei FatFs eine sehr gute, konstante Performance von 5 ms bei 8 MHz liefert, zeigt sich bei littlefs der Overhead der Datenintegritätskontrolle: Grundsätzlich ist der Schreibzyklus beim littlefs-Dateisystem ca. 8 mal langsamer. Das weitaus fatalere Verhalten ist aber, dass das Anhängen von Daten in regelmäßigen Abständen deutlich länger braucht. Die zusätzliche Zeit, die es benötigt, ist abhängig von der Dateigröße. Bei einer Dateigröße von nur 1 MB dauert diese zusätzliche Erstellzeit bereits über 10 s. Eine bessere Implementierung und größere Buffer in der Konfiguration können diese Zahlen quantitativ zwar reduzieren, ändern aber nichts an ihrem prinzipiellen Vorhandensein. Das macht littlefs als Echtzeitdateisystem unbrauchbar. Im Hintergrund scheint littlefs für seine Dateiintegritätsmechanismen immer wieder die gesamte Datei zu überprüfen, was für einen Einsatz in embedded Umgebungen zu viele Hardwareressourcen bindet. Dateisysteme wie ext4 nutzen ihr Journaling beispielsweise bewusst nur für ihre Metadaten, um die Dateisystemintegrität abzusichern, damit es nicht zu schweren Performanceeinbußen kommt wie sie hier zu beobachten sind.

Fazit Versuch 3

5.5 Versuch 4: Verhalten der Dateisysteme bei Stromunterbrechungen

Der vorhergehende Versuch war im Grunde genommen eine Vorbereitung für ein gutes Design von diesem Versuch. Die Entwickler des littlefs-Dateisystems versprechen eine hohe Datei- und Dateisystemkonsistenz bei Stromausfällen. Dies soll praktisch getestet werden und mit dem FAT-Dateisystem verglichen werden. Eine erste Idee war die `watchdog`-Funktionalität des STM32G431 zu nutzen. Dieses greift allerdings zu kurz, da dabei keine Unterbrechung der SD-Karten-Spannungsversorgung herbeigeführt wird.

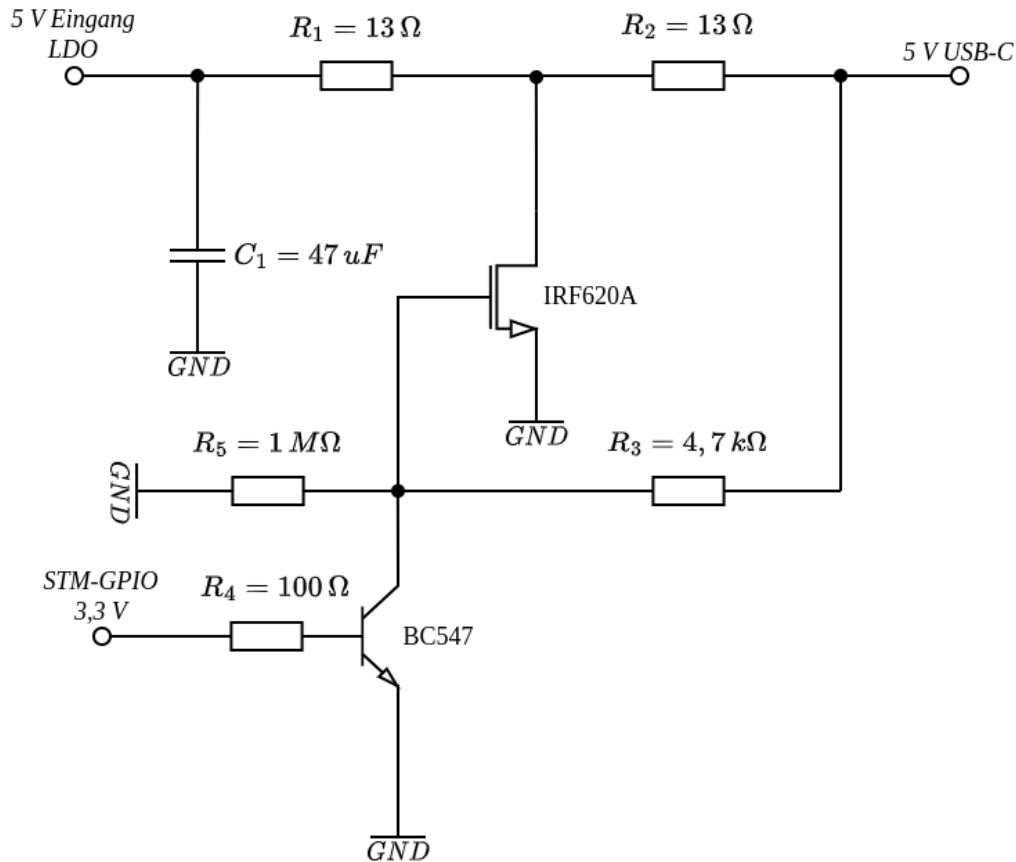


Abbildung 28: „killswitch“-Schaltung für eine periodische Unterbrechung der Spannungsversorgung

Die Spannungsversorgung der in dieser Bachelorarbeit verwendeten Platine wird entweder über die vorhandene 5V USB-C-Buchse oder über eine Stiftleiste, die hauptsächlich für ein Batteriefach mit einer variablen Spannung zwischen 3,8 und 5,5 V gedacht ist, gewährleistet. Welche der beiden Optionen verwendet wird entscheidet ein Jumper. An dieser Stelle lässt sich praktischerweise eine externe Schaltung einklinken, die periodisch die Spannungsversorgung unterbricht.

Abbildung 28 zeigt die für diesen Versuch erstellte Schaltung. Leider musste diese aufgrund der vorhandenen Bauteile etwas komplexer gestaltet werden als im Prinzip notwendig. Letztlich braucht es lediglich einen Schalter, der periodisch 5 V an- und ausschaltet. So wäre ein mit 3,3 V schaltbares Relais mit einer Rücklaufdiode ausreichend für diesen Zweck gewesen. Aufgrund von eingeschränkter Bauteilverfügbarkeit wurde aber auf den vorhandenen MOSFET IRF620A zurückgegriffen. Dieser kann im durchgeschalteten Zustand den Pegel vom 5 V LDO Eingang auf nahezu 0 V bringen. Das ist ohne destruktiven Kurzschluss natürlich nur durch die Widerstände R_1 und R_2 möglich. Diese begrenzen den für den LDO verfügbaren Strom auf rund $\frac{5V}{26\Omega} \approx 190\text{ mA}$. Dies ist für die durchschnittliche Stromaufnahme der Platine (gemessen: 23,3 mA) vollkommen ausreichend und wird mit dem Kondensator C_1 zusätzlich gepuffert. Gleichzeitig kann der verwendete MOSFET den Stromfluss problemlos gegen GND abführen, sodass die Spannung auf 0 V einbricht. Unter der Annahme, dass der MOSFET im durchgeschalteten Zustand keinen Innenwiderstand besitzt, fließt einerseits ein Strom von $\frac{5V}{13\Omega} \approx 385\text{ mA}$ vom USB-C Zweig, andererseits ebenfalls maximal 385 mA vom Zweig mit dem Kondensator C_1 . Da der MOSFET bis zu 5 A schalten kann ist dies

im sehr sicheren Bereich für das Bauteil.

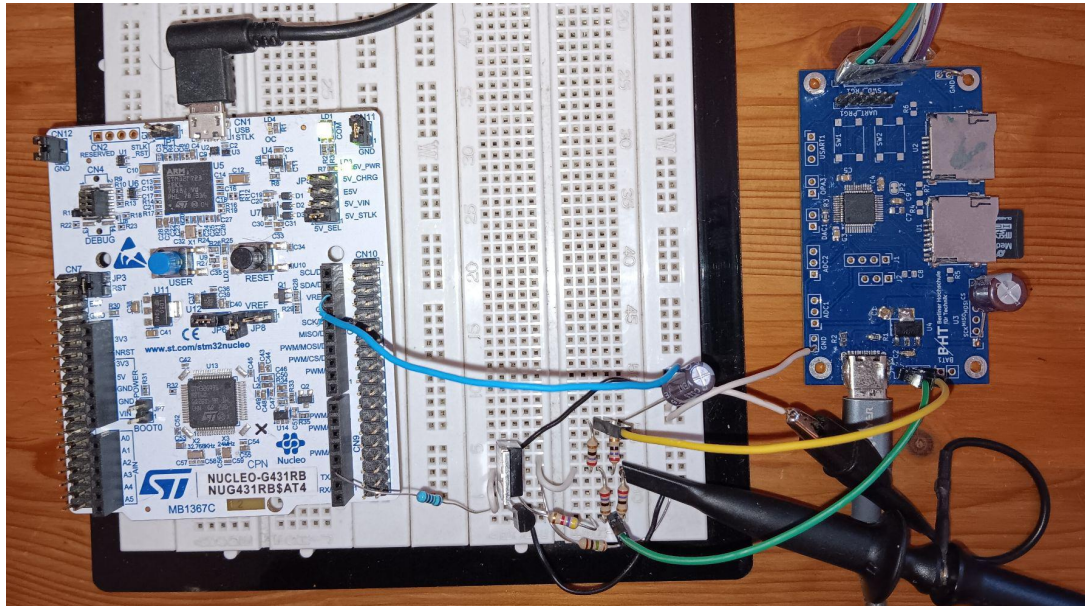


Abbildung 29: Aufbau der „killswitch“-Schaltung auf einem Steckbrett

Die Idee war zunächst den MOSFET mit einem GPIO-Pin eines externen Nucleo-Boards zu schalten. Leider schaltet der IRF620A erst zuverlässig bei $U_{gs} = 5\text{ V}$ durch, sodass eine zusätzlicher NPN-Transistor in Emitterschaltung hinzugefügt werden musste, der die 5 V der USB-C Buchse nutzt. Dieser kann dann mit dem GPIO-Pin eines beliebigen Nucleo-Boards geschaltet werden.

```
1  GPIO_Init();
2  TIM2_Init();
3
4  GpioPin<GPIOA_BASE,10>::OutputInit(Level::High);
5  for(;;)
6  {
7      //ON for 10 secs
8      GpioPin<GPIOA_BASE,10>::setHigh();
9      delay_s<10>();
10     //OFF for 3 secs
11     GpioPin<GPIOA_BASE,10>::setLow();
12     delay_s<3>();
13 }
```

Quellcode 16: Hauptschleife aus Datei src/killswitch.cpp

Quellcode 16 zeigt praktisch den gesamten Quelltext der Datei src/killswitch.cpp, der aufgrund der bereits vorhandenen Abstraktionen sehr einfach ausfällt. Für 10s bleibt der GPIO-Pin auf einem 3,3V Pegel und erlaubt der Dateisystem-Platine den Schreibtest auszuführen. Dann wird der Pin auf einen 0V Pegel gesetzt und simuliert der Platine einen plötzlichen Stromausfall. Es finden in diesem Versuch also ca. 4

Stromausfälle pro Minute statt.

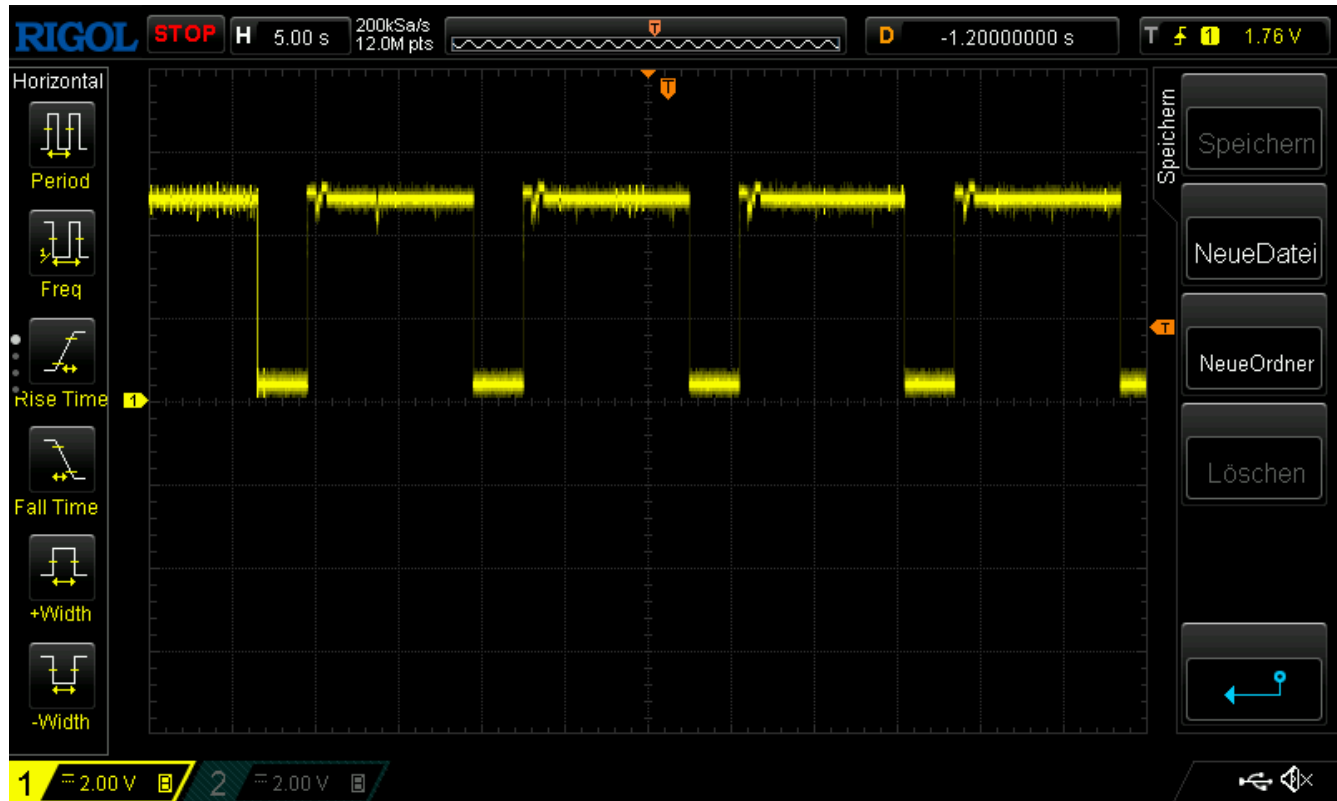


Abbildung 30: Verifizierung der Spannung am LDO-Eingang

Abbildung 30 verifiziert die korrekte Spannungsversorgung am LDO-Eingang. Es fällt zusätzlich auf, dass die Schaltung etwas Rauschen eingebracht hat, was für diesen Versuch, der Schreibfehler provozieren möchte, grundsätzlich positiv zu bewerten ist.

Die verwendeten Programme für den Test sind `test/fatfs_killswitch_test.cpp` und `test/littlefs_killswitch_t`. Der Test an sich nutzt wieder eine für Menschen lesbare, verifizierbare Zeichenfolge. Diese ist bei Fehlern leichter Handzuhaben als die pseudo-randomisierte Folge aus dem vorherigen Versuch. Das Programm prüft zunächst wie viele Testdateien bereits existieren und erstellt dann die nächst größere Datei. Grundsätzlich wird dann eine bekannte Zeichenfolge bis zum Stromausfall an die erstellte Datei immer wieder angehängt. Die Größe der Dateien hängt also hauptsächlich davon ab, wie schnell die Dateisysteme den nächsten freien Dateinamen finden und wie schnell sie an diese Datei anhängen können.

Die SPI-Clock wurde auf das maximale Tempo von 8 MHz gesetzt. Es wurden für diesen Test rund 140 MB FatFs-Daten und 2,1 MB an littlefs-Daten erzeugt. Die verhältnismäßig geringe Menge an littlefs-Daten hängt damit zusammen, dass das Dateisystem auch beim Suchen der Dateien deutlich langsamer ist als FatFs und es somit innerhalb der vorgegebenen 10 Sekunden Versuchslaufzeit maximal 52 Dateien erstellen konnte. Alle weiteren Versuche beanspruchen schlichtweg mehr als 10 Sekunden für die Suche des nächsten freien Dateinamens und die Erstellung einer neuen Datei. Der Algorithmus ließe sich natürlich optimieren, sodass noch mehr littlefs-Testdaten erzeugt werden können. Tatsächlich zeigen allerdings die Testdaten des unsichereren Dateisystems FatFs, dass sich mit Stromausfällen auf SD-Karten praktisch

keine Schreibfehler erzeugen lassen. Der Effekt war bei FatFs im Übrigen ebenfalls vorhanden. Dennoch konnte problemlos die 70-fache Menge an Daten in derselben Zeit erzeugt werden. Das Pythonskript zur Verifikation findet sich unter [\(root\)/test_data/Writetest4/test_readable.py](#).

Fazit Versuch 4

Trotz der kontinuierlichen Stromausfälle kam es in keinem der beiden Dateisysteme auch nur zu einem Schreibfehler. In Kombination mit den vorangegangenen Versuchen lässt sich somit behaupten, dass Schreibfehler im üblichen SD-Kartenbetrieb sehr selten sind und selbst bei exzessiven Stromausfällen nicht häufig vorkommen: Nach der statistischen Dreier-Regel (Rule of Three) lässt sich bei N fehlerfreien Übertragungen die maximale Bitfehlerrate (BER) für eine Sicherheit von 95 % wie folgt grob abschätzen:

$$\text{BER}_{95\%} \leq \frac{3}{N}$$

Aufgrund von 140 MB an fehlerfreien Daten folgt dann $N = 140 \cdot 1024 \cdot 1024 \cdot 8$ Bits.

$$\text{BER}_{95\%} \leq \frac{3}{140 \cdot 1024 \cdot 1024 \cdot 8} \approx 2,55 \cdot 10^{-9}$$

Der Versuch schätzt also eine maximale Bitfehlerrate in der Größenordnung von 10^{-9} bei exzessiven Stromausfällen und fehlender CRC-Prüfsumme auf SD-Übertragungsebene. Der in die Karte integrierte SD-Kartencontroller scheint also bereits dafür zu sorgen, dass auch bei Stromausfällen nur verlässliche Daten auf der Karte landen. Für eine zusätzliche Sicherheit bei der Übertragung ließe sich die CRC-Prüfsumme, die im SPI-Modus standardmäßig deaktiviert ist, wieder aktivieren.

Der Versuch hat ebenfalls noch einmal verdeutlicht, dass der Umgang mit littlefs in Echtzeitsystemen sehr genau geplant sein muss. Für eine Einhaltung von gegebenen Zeitgrenzen müssen bei dem Dateisystem vorab Tests und Messungen gemacht werden, während FatFs sich responsiver und weniger unberechenbar verhält.

Das endgültige Fazit ist hier, dass der zusätzliche Datenschutz durch das littlefs-Dateisystem die höhere Komplexität des Systems im Zusammenhang mit SD-Karten nicht rechtfertigt.

5.6 Versuch 5: Direkte Blockadressierung (NoFs)

Die vorangegangenen Kapitel untersuchten das Verhalten von littlefs und FatFs in unterschiedlichen Szenarien. Angedacht war auch die Untersuchung von direkter Blockadressierung. Dies hat sich aber aus mehreren Gründen als nicht sinnvoll herausgestellt: Einerseits nutzen beide Dateisysteme die Funktionen der Blockadressierung als Basis. Wenn es FatFs möglich ist fehlerfrei Dateien zu erstellen, so prüft das indirekt ein korrektes Verhalten der SD-Kartenblockschreibfunktionen.

Andererseits ist der Aufwand nicht ganz unerheblich: Während FatFs und littlefs im Prinzip ähnliche Funktionen zum Öffnen, Schreiben und Schließen bereitstellen, so muss für die direkte Blockadressierung eine eigene Struktur erfunden werden, in der die Daten abgelegt werden. Ein ganz direkter Vergleich ist damit auch nicht möglich.

Am ehesten lässt sich Teil 1 von [Versuch 3](#) wiederholen, der die Zeitdauer des Dateianhangs von 1 KB Daten misst. Das ist insofern inhaltlich interessant, weil es aufzeigt wie viel Zeit das eigentliche Schreiben von Blöcken mit den gegebenen Funktionen benötigt und wie viel Zeit dem Overhead des Dateisystems zuzuordnen ist.

Gemessen wird die Dauer für das Schreiben eines Blocks (512 Bytes) und die Dauer des Schreibens für zwei Blöcke (1024 Bytes). Das Schreiben von mehreren Blöcken nutzt ein unterschiedliches SD-Kommando und sollte zeitmäßig dem Schreiben von einzelnen Blöcken überlegen sein. Auf einem Linuxsystem lassen sich die Rohdaten wiederum mit

`dd if=/dev/sdx of=sd_karte_dump.bin bs=512 count=Anzahl_Bloেকে status=progress` extrahieren.

Die Versuchsquelldateien findet sich in `test/nofs_append_write_test_512.cpp` und `test/nofs_append_write_test_1024.cpp`.

Die aufgenommenen Testdaten sind unter `(root)/test_data/Writetest5` erreichbar. Der SPI-Clock wurde auf 1 MHz belassen.

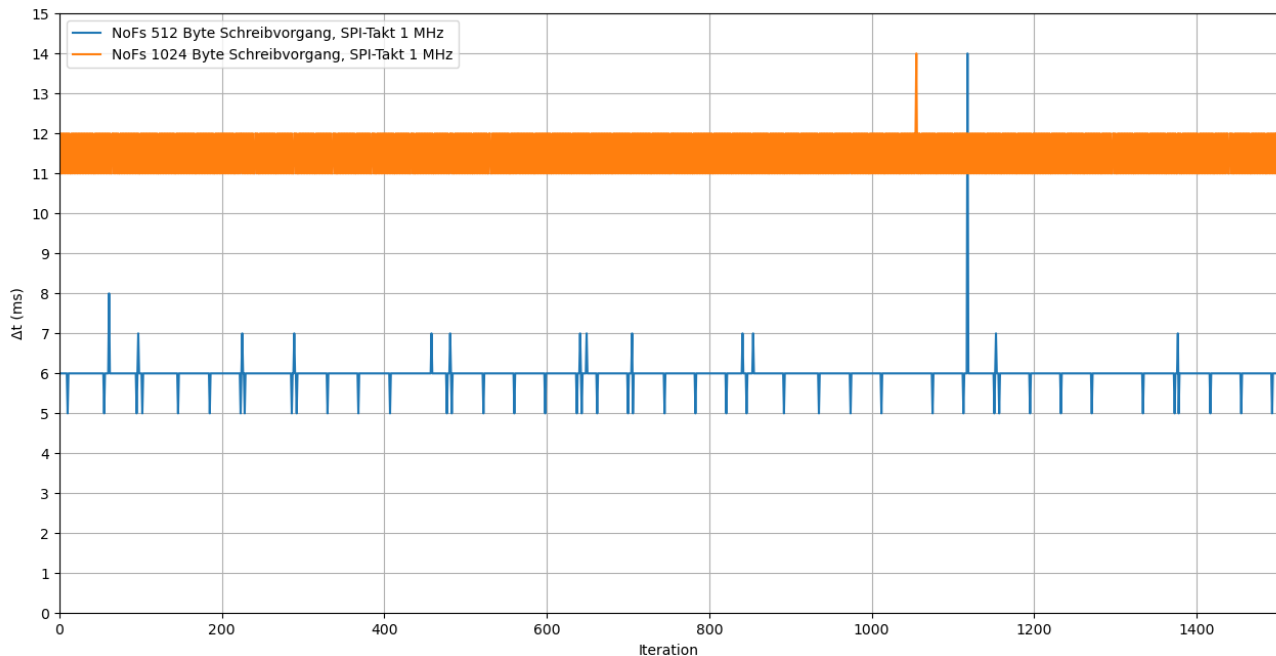


Abbildung 31: Dauer von 512 Byte und 1024 Byte Schreibvorgängen

Abbildung 31 plottet die aufgenommenen Daten. Dabei war bereits zu erwarten, dass das Schreiben eines 512 Byte Blocks in etwa doppelt so lange in Anspruch nimmt als das Schreiben von zwei 512 Byte Blöcken. *Tabelle 6* nimmt den Durchschnitt der ersten 1000 Werte für die Schreibdauer von 1024 Bytes. Für NoFs (Einzelblock) wurde die aufgenommene Dauer verdoppelt. Interessant ist hierbei auch ein Vergleich der Daten von littlefs (SPI 1 MHz) und FatFs (SPI 1 MHz) aus [Versuch 3](#). Tatsächlich zeigt sich, dass FatFs in seiner Performance sogar besser abschneidet als NoFs bei Nutzung der Einzelblockschreibfunktion. Es führt beim Abhängen an eine Datei im Prinzip gar keinen Schreibzeitoverhead ein. Der Unterschied zwischen dem Schreiben von zwei Blöcken und dem Schreiben von einem Einzelblock beträgt lediglich 0,5 ms.

Dieser würde aller Voraussicht nach beim Schreiben von noch mehr Blöcken gleichzeitig mehr ins Gewicht fallen. Dadurch, dass in den allermeisten Fällen die zu schreibenden Informationen in einen Schreibbuffer gelegt werden müssen, können in ressourcenbeschränkten Umgebungen aber nicht beliebig viele Blöcke gleichzeitig geschrieben werden.

Tabelle 6: Durchschnittliche Dauer des Schreibvorgangs für 1024 Bytes

Dateisystem	Δt 1024 Bytes	Datenrate
NoFs (Einzelblock)	11,96 ms	85618,72 B/s
NoFs (Multiblock)	11,48 ms	89198,61 B/s
FatFs SFN	11,74 ms	87223,17 B/s
littlefs	202 ms	5069,31 B/s

Zeitmäßig deutlich abgehangen ist littlefs, wobei die Durchschnittsbildung in diesem Fall etwas irreführend ist: Grundsätzlich dauert das Anhängen von 1 KB bei littlefs um die 80 ms. Der Durchschnitt wird aber durch fatale Ausreißer, die mehrere Sekunden dauern können, stark gesenkt.

Die ideale Datenrate bei dem verwendeten SPI-Takt von 1 MHz beträgt 125000 B/s. In allen Fällen ist beim Design die Erstellung des 1024 Byte Datenbuffers mitgemessen worden, sodass die eigentliche Datenrate sogar ein klein wenig höher ausfällt. Zudem muss bedacht werden, dass hier lediglich die Geschwindigkeit des Payloads gemessen wurde. Das SD-Protokoll selbst hat einen Overhead für die SD-Kommandos, zusätzlich entstehen teilweise Wartezeiten, weil die SD-Karte noch vorangegangene Operationen zuende führen muss.

FatFs und NoFs befinden sich in ihrer Schreibperformance also erstaunlich nah am physikalischen Limit der SPI-Peripherie.

Fazit Versuch 5

Das Fazit in diesem Versuch ist die Erkenntnis, dass FatFs bei der genutzten Konfiguration und den vorhandenen Blockschreibfunktionen keinen Schreiboverhead einführt. Da bereits festgestellt wurde, dass FatFs auch hinsichtlich der RAM-Nutzung kaum schwerer wiegt als NoFs, ist der einzige größere Nachteil von FatFs gegenüber NoFs der Flashverbrauch, der im schlimmsten Fall rund 13 KB höher ausfällt.

6 Gesamtfazit

Die Hardware war für einen ersten Prototypen gut geeignet und funktional. Alle Schreib- und Leseversuche ließen sich damit zunächst einmal durchführen. Für zukünftige Produkte müsste der beschriebene Spannungseinbruch in der MISO-Leitung bei einer der zwei SPI-Peripherien näher untersucht werden.

Auf der Softwareseite hat sich das Setup aus CMake, VsCode, Cortex-Debug und C++ bewährt: Dank CMake konnten bei Änderungen in den Haupt Quelldateien zahlreiche Binaries zur selben Zeit automatisiert kompiliert werden. Da das System nur veränderte Dateien kompiliert, war die Kompilierzeit sehr niedrig und die Iterationsgeschwindigkeit für neue Funktionen sehr hoch. Während die weitaus komplexere Syntax von C++ im Vergleich zu C abschreckend ist, bieten moderne Funktionalitäten mehr Möglichkei-

ten Abstraktion, Wiederverwendbarkeit und Erweiterbarkeit für zukünftige Projekte. Da viel Abstraktion mit modernem C++ in die Kompilierzeit verschoben werden kann, bleibt die Ausführungsgeschwindigkeit trotz Nutzung von (eigenen) Libraries wie der `src/GPIO_HAL.hpp` sehr hoch.

Die Arbeit am SD-Treiber war eine der Grundvoraussetzungen für den Rest der Arbeit. Natürlich implementiert diese nicht das gesamte SD-Protokoll, sondern nur die für diese Arbeit relevanten Funktionen: Das Schreiben und Lesen von Einzel- und Mehrfachblöcken und das Abrufen von SD-Karteninformationen. Es konnte festgestellt werden, dass die Schreibperformance sich nah an der physikalischen Grenze der SPI-Peripherie befindet und sehr zuverlässig funktioniert. Tatsächlich ließ sich über alle Versuche hinweg kein einziger Schreibfehler provozieren und es musste für eine Untersuchung von Schreibfehler eine gezielte Injektion von falschen Daten erfolgen.

Ein weiterer Vorteil des SD-Treibers ist der recht geringe Flashspeicherverbrauch von nur rund 2 KB, so dass er sich problemlos in eine Vielzahl von weiteren aufbauenden Projekten integrieren lässt. Ein großer Teil der Arbeit bestand in dem Vergleich zwischen FatFs und littlefs. Noch einmal sollen an dieser Stelle die drei großen Vorteile des littlefs-Dateisystems genannt werden:

- **Ausfallsicherheit bei Stromausfall**
- **Verschleißregulierung des Flashspeichers (Wear Leveling)**
- **Begrenzter RAM/ROM-Bedarf**

Leider fallen im Zusammenhang mit SD-Karten die Vorteile sehr gering aus: [Versuch 4](#) hat gezeigt, dass die tatsächlichen Bitfehlerraten beim Schreiben und der Übertragung extrem niedrig sind. Durch Nutzung des SD-Karteninternen CRC-Prüfsumme besteht optional bereits auf SPI/SD-Kartenebene ein zusätzlicher Fehlerschutz. Gleichzeitig führt das integrierte Fehlerprüfsystem zu einem Verlust von leicht vorhersagbarer Echtzeitfähigkeit des Systems, da das Dateisystem in regelmäßigen Abständen deutlich länger zum Schreiben einer Datei braucht als im Durchschnitt.

Verschleißregulierung des Flashspeichers ist hinfällig, da der SD-Kartencontroller diese Aufgabe bereits erledigt. Während dies in langen experimentellen Reihen getestet werden müsste, bietet die zusätzliche Verschleißregulierung auf Dateisystemebene wahrscheinlich keine zusätzlichen positiven Effekte. Vielmehr scheint es wahrscheinlich, dass das zusätzliche Journaling zu mehr Schreiboperationen führt und damit potentiell die SD-Karte belastet.

Auch beim begrenzten RAM/ROM-Bedarf verliert littlefs im direkten Vergleich mit FatFs aufgrund seines komplexeren Aufbaus. Abschnitt [5.1](#) zeigt die Größenunterschiede auf. Während littlefs sich vom Speicher her problemlos auf moderne Mikrocontroller programmieren lässt, hat FatFs in einer gewöhnlichen Standardkonfiguration lediglich die halbe Programmgröße und lässt sich durch die Einschränkung von Optionen in der `FatFs/ffconf.h` sogar noch weiter verkleinern.

Dennoch hat [Versuch 2](#) klargemacht, dass die Verwendung von littlefs bei fehlerbehaftetem Nachrichtenübertragungskanal durchaus von Vorteil sein kann, wenn man die erwähnten Nachteile beachtet: Während FatFs schnell zu Inkonsistenzen im Dateisystem, Metadaten und Dateien selbst neigt, demonstriert littlefs eindrucksvoll, dass die Mechanismen zur Erkennung von fehlerhaften Daten selbst bei hohen Bitfehlerraten einwandfrei funktionieren.

Alles in allem gewinnt in der Praxis allerdings die Verwendung von FatFs bei SD-Karteneinsatz, selbst

wenn es auf Datenintegrität ankommt: Da das FAT-Dateisystem seit über 40 Jahren in Verwendung ist, existiert sehr viel Software wie beispielsweise „fsck“, die fehlerhafte Dateisystem- und Datenintegrität teilweise wiederherstellen kann. Auch wenn es in dieser Arbeit nicht demonstriert wurde, so lässt sich mit CRC-Prüfsummen auf SPI/SD-Kartenebene ohne einen erheblichen Mehraufwand zusätzliche Sicherungsschicht einführen, die im Gegensatz zu littlefs die Schreibgeschwindigkeit weiterhin mit konstanter Laufzeit beibehält. Fehlerschutz muss weiterhin nicht auf Dateisystemebene passieren: Aufbauend auf FatFs lassen sich externe Schutzmechanismen wie die bereits genannten CRC-Prüfsummen oder kryptographische Hashfunktionen wie SHA-256 oder MD5 anwenden, wie sie auch bei Downloads von größeren Dateien üblich sind. Mit Datensicherungsstrategien wie doppelter oder dreifacher Redundanz lässt sich ohne zusätzlichen Rechenaufwand zusätzliche Sicherheit schaffen mit dem Preis des Speicherplatzes. Die Verwendung von reinem Blockspeicher hingegen (NoFs) hat sich recht schnell als zu aufwendig herausgestellt: Ohne die Herstellung eines eigenen Systems zur Speicherung der Daten ist das Wiederauffinden einer bestimmten Information sehr schwierig. Weiterhin müsste ein Mechanismus implementiert werden, der die Daten extrahiert und letztlich wieder in eine dateisystemlesbare Datei überführt. NoFs eignet sich daher höchstens für sehr einfache Daten, die einem simplen Schema folgen, wie beispielsweise Temperatursensordaten mit einer festen Größe. Ein mögliches Vorgehen für NoFs wurde in [Versuch 3](#) demonstriert.

Alles in allem liegt der Verwendungszweck von littlefs vor allem bei reinem Flashspeicher. Aber ganz unabhängig vom physikalischen Medium: Sollte allerdings eine Lösung für die irregulären, sehr langen Schreibvorgänge auftauchen, so wäre littlefs eine echte Alternative bei störanfälligen Nachrichtenkanälen.

Quellenverzeichnis

- [1] Tanenbaum, A. S. & Bos, H. (2015). *Modern Operating Systems* (4. Aufl.). Vrije Universiteit Amsterdam, The Netherlands: Prentice Hall.
- [2] STMicroelectronics. (2026). *STM32G431RB Mainstream Arm Cortex-M4 MCU Product Page*. Offizielle Produkt Information. Abgerufen von <https://www.st.com/en/microcontrollers-microprocessors/stm32g431rb.html> (Besucht am 26. Juli 2026).
- [3] STMicroelectronics. (2021). *STM32G431x6 STM32G431x8 STM32G431xB Datasheet (DS12589 Rev 6)*. Offizielle Spezifikation. Abgerufen von <https://www.st.com/resource/en/datasheet/stm32g431rb.pdf> (Besucht am 26. Juli 2026).
- [4] STMicroelectronics. (2025). *RM0440 Reference manual: STM32G4 series advanced Arm-based 32-bit MCUs (Rev 9)*. Offizielle Spezifikation. Abgerufen von https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf (Besucht am 26. Juli 2026).
- [5] Ababei, C. (2013). *Lecture 12: SPI and SD cards*. Vorlesungsskript, EE-379 Embedded Systems and Applications, University at Buffalo. Abgerufen von https://www.dejazzzer.com/ee379/lecture_notes/lec12_sd_card.pdf (Besucht am 6. Juli 2026).

- [6] SD Group Matsushita Electric Industrial Co., Ltd. (Panasonic) (2006). *SD Specifications Part 1: Physical Layer Simplified Specification, Version 2.00*. Technische Spezifikation, SD Card Association. Abgerufen von https://users.ece.utexas.edu/~valvano/EE345M/SD_Physical_Layer_Spec.pdf (Besucht am 8. Juli 2026).
- [7] ChaN (2024). *FatFs - Generic FAT Filesystem Module*. Software-Dokumentation, Electronic Lives Manufacturing. Abgerufen von <https://elm-chan.org/fsw/ff/> (Besucht am 23. Juli 2026).
- [8] ChaN (2020). *FAT Filesystem*. Software-Dokumentation, Electronic Lives Manufacturing. Abgerufen von https://elm-chan.org/docs/fat_e.html (Besucht am 24. Juli 2026).
- [9] Microsoft (2023). *File System Functionality Comparison*. Software-Dokumentation, Microsoft Learn. Abgerufen von <https://learn.microsoft.com/en-us/windows/win32/fileio/filesystem-functionality-comparison> (Besucht am 24. Juli 2026).
- [10] Microchip Technology Inc. (2024). *How to interface a microSD media with Microchip SD/MMC Card Readers*. Support Knowledge Base. Abgerufen von <https://support.microchip.com/s/article/How-to-interface-a-microSD-media-with-Microchip-SD-MMC-Card-Readers> (Besucht am 8. Juli 2026).
- [11] UEFI Forum. (2024). *UEFI Specification Version 2.11 (Chapter 5: GUID Partition Table Format)*. Official Specification. Abgerufen von https://uefi.org/specs/UEFI/2.11/05_GUID_Partition_Table_Format.html (Besucht am 24. Juli 2026).
- [12] The Linux Kernel Documentation (2026). *Journal (jdb2)*. Offizielle Kernel Dokumentation. Abgerufen von <https://www.kernel.org/doc/html/latest/filesystems/ext4/journal.html> (Besucht am 17. August 2026).
- [13] Michael Eiler, *C++20: Building a Thread-Pool With Coroutines* <https://blog.eiler.eu/posts/20210512/>
- [14] Microchip Technology Inc. (2021). *How to interface a microSD media with Microchip SD/MMC Card Readers*. Abgerufen von <https://support.microchip.com/s/article/How-to-interface-a-microSD-media-with-Microchip-SD-MMC-Card-Readers> (Besucht am 14. August 2026).
- [15] littlefs-project. (2026). *GitHub - littlefs-project/littlefs: A little fail-safe filesystem designed for microcontrollers*. Offizielles Quellcode-Repository. Abgerufen von <https://github.com/littlefs-project/littlefs> (Besucht am 17. August 2026).